

# Data-Driven Computer Animation

## Lecture 2.

### Basics of Character Animation

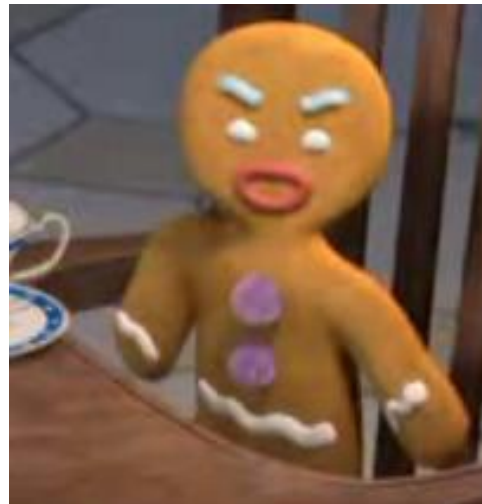
Taku Komura

# Overview

- Character Animation
  - Posture representation
    - Hierarchical structure of the body
    - Joint types
      - Translational, hinge, universal, gimbal, free
    - Euler angles
      - Gimbal lock
    - Quaternions
- Creating the animation (Keyframe animation)
  - Forward Kinematics
  - Interpolation
  - More about Inverse Kinematics
    - Analytical
    - CCD
    - Pseudo Inverse

# Characters include

- ☐ Human models
- ☐ Virtual characters
- ☐ Animal models



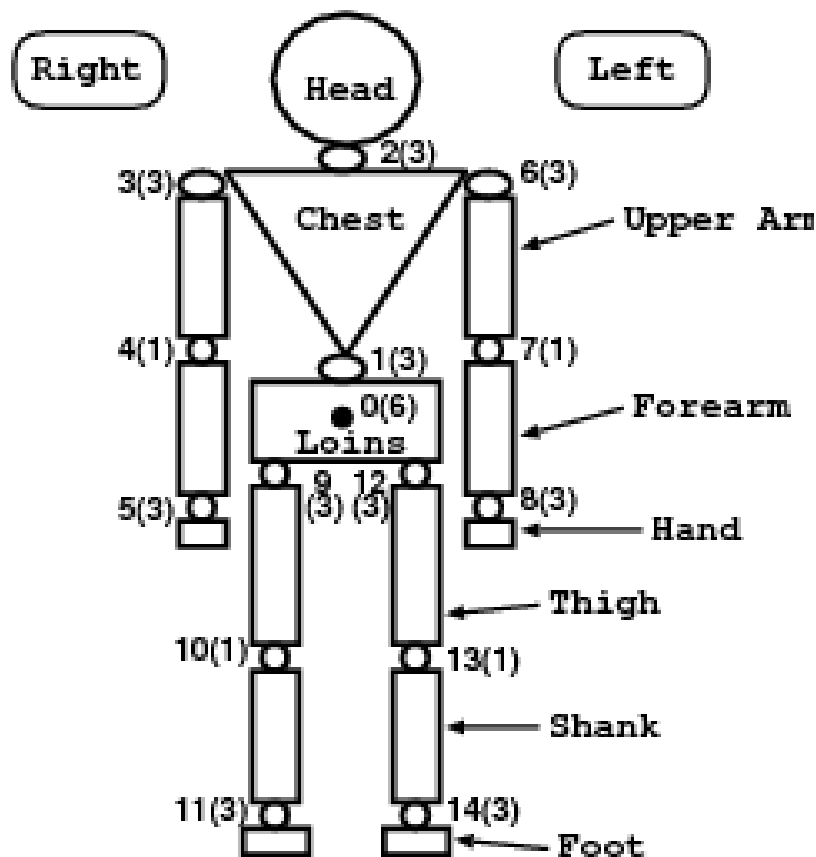
# Controlling the Skeleton

- We control the skeleton of the character  
→ *skeletal motion*
- The skin follows the movement of the skeleton  
→ *skinning deformation*

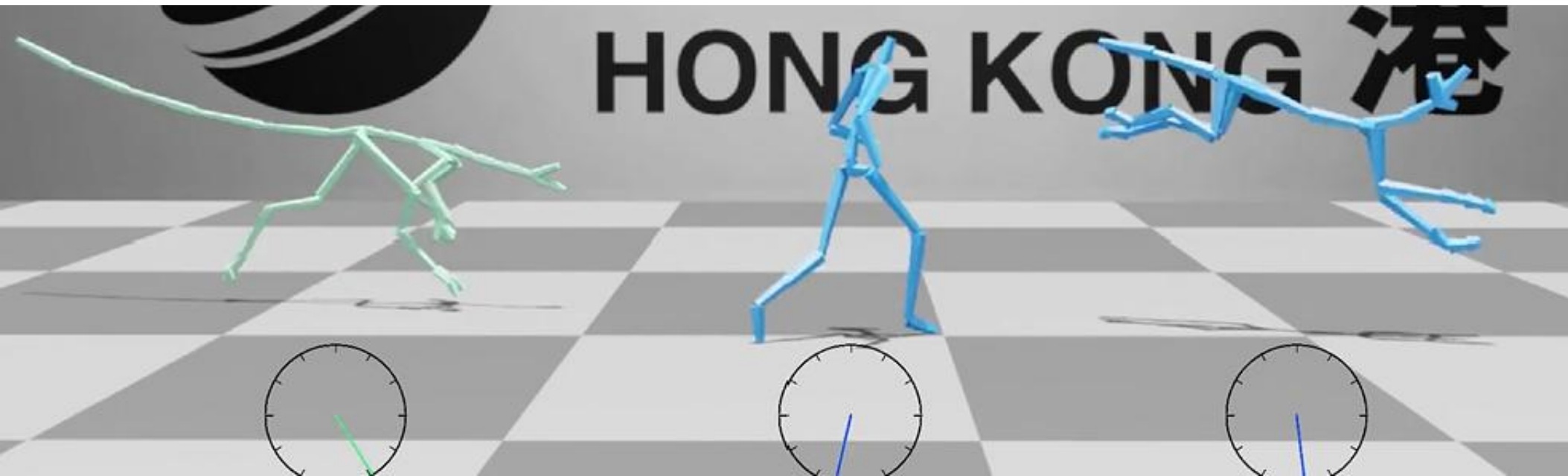


# Representation of Postures

- The body has a hierarchical structure
- Many & different types of joints



# Different Bodies Have Different Skeletons



# Hierarchical structure of the body

- The first joint is typically called the “root” of the character
- The position of the joints lower in the hierarchy are affected by those above it
- Each joints can have 1 to 6 degrees of freedom (DOF)
- For rotational joints, usually it is 1, 2, or 3
- The root of the body has 6 DOF in total (3 DOF for translation, 3DOF for rotation)

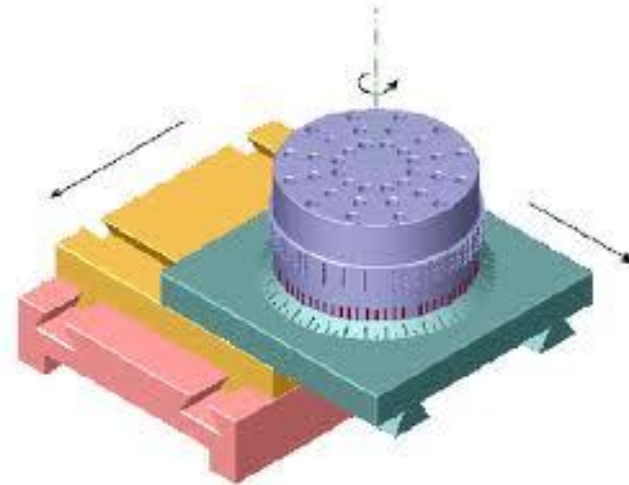
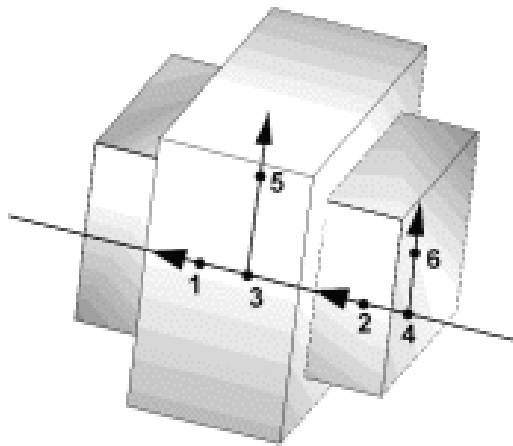


# Joints

- The Degrees of Freedom (DOF) is defined for various joints
- There are several kinds of joints
  - Translational joint (1,2,3 DOF)
  - Hinge joints (1 DOF)
  - Universal joint (2 DOF)
  - Gimbal joint (3 DOF)
  - Free joint (3 DOF)

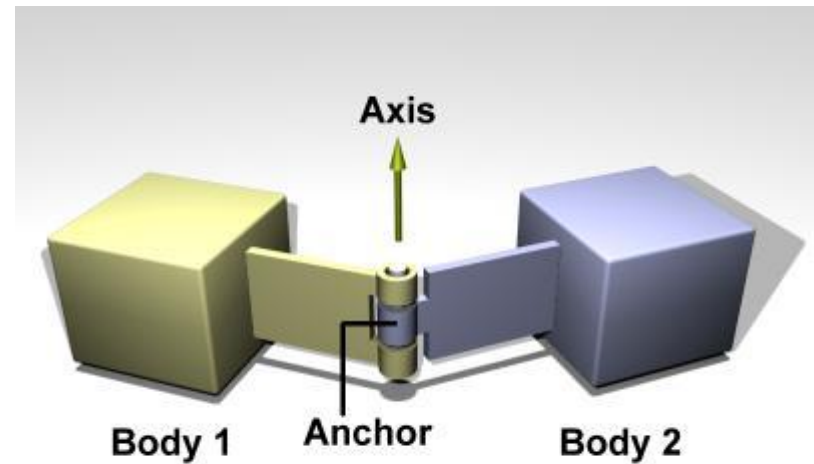
# Translational joint

- A sliding joint
- Can be 1, 2 or 3 DOF



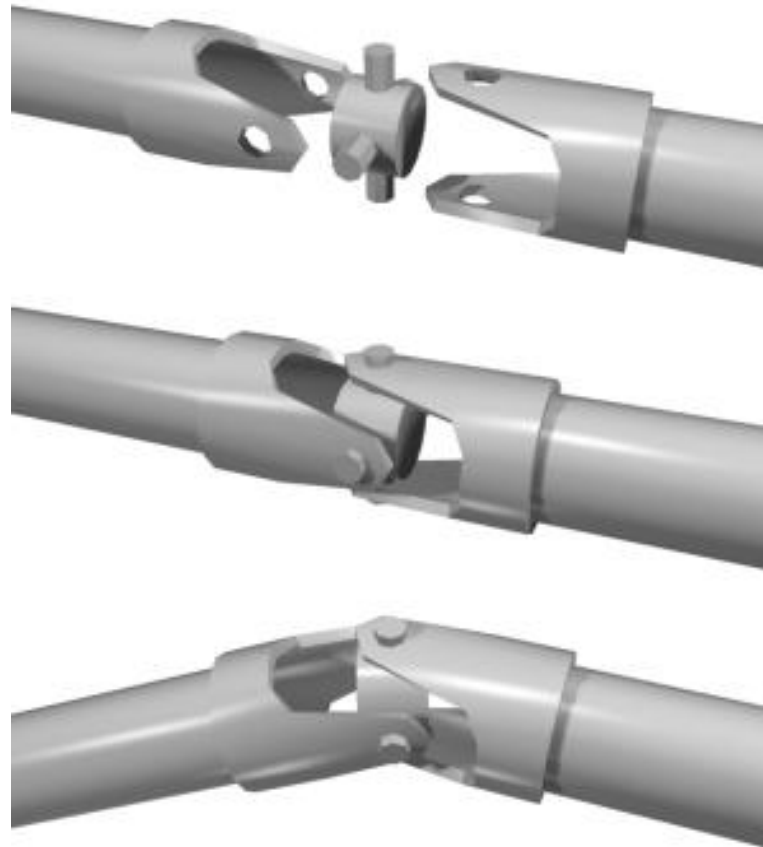
# Hinge Joint

- A 1 DOF rotational joint
- Can be defined by the axis of rotation
- **Knee, elbow**



# Universal Joint

- ❑ 2DOF
- ❑ Rotation around 2 axes perpendicular to each other
- ❑ **Wrist joint**



# 3DOF rotational joints

- Shoulder, hip, neck
- A few ways to represent the rotations
  - Rotation matrix
  - Gimbal joint (Euler Angles)
  - Free joint (Quaternions)
  - 6D representation

# Rotation Matrices

$$R = \begin{pmatrix} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{pmatrix} \quad \begin{array}{l} u = (u_0, u_1, u_2)^T, v = (v_0, v_1, v_2)^T, \\ w = (w_0, w_1, w_2)^T \in \mathbb{R}^3 \\ \text{orthonormal basis} \end{array}$$

Nine numbers needed for a rotation

New rotations are obtained by matrix-matrix multiplication; vectors are rotated by matrix-vector multiplication

But a bit memory intensive

# Gimbal joint: Euler Angles

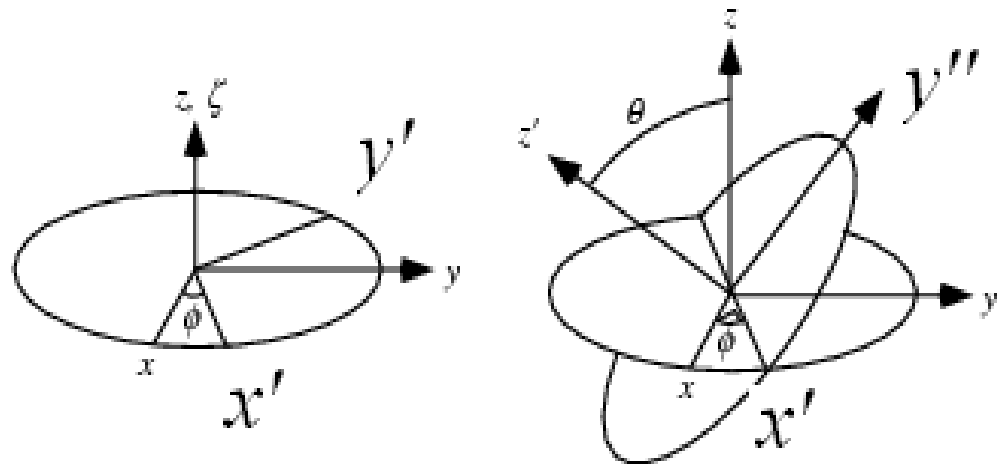
- ❑ 3DOF joints
- ❑ Comes from Robotics
- ❑ 3DOF joints in robots were designed by connecting three motors pointing different axes



# Gimbal joint: Euler Angles

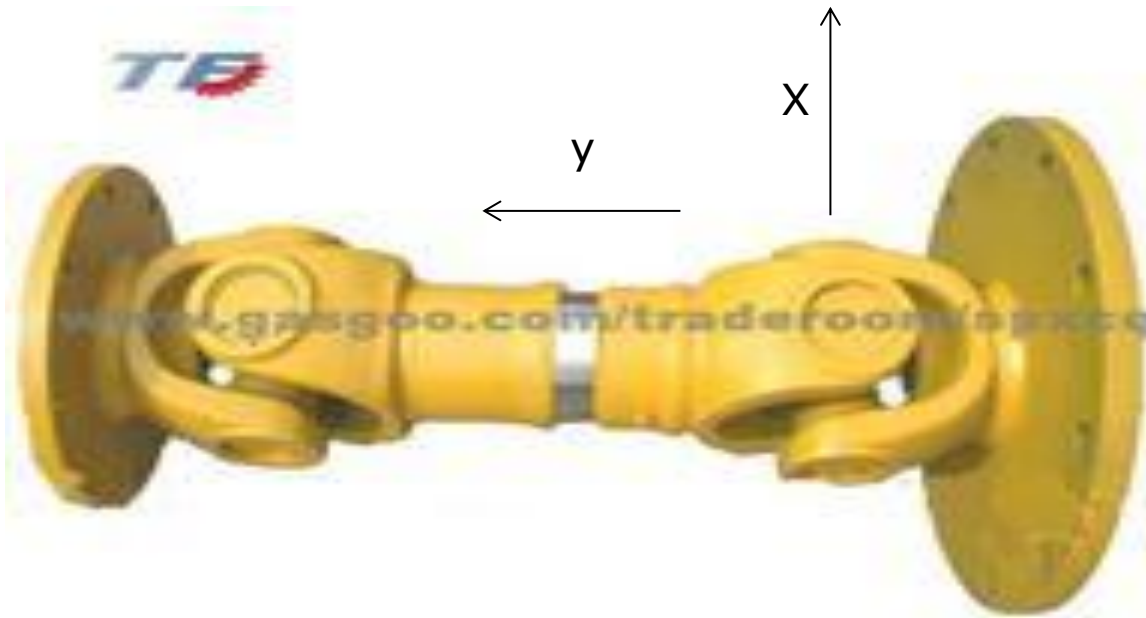
- ❑ Rotation defined by the three axes and the angle of rotation around them
- ❑ the rotation order has to be specified such as X-Y-Z, Z-X-Y, Y-Z-X, etc

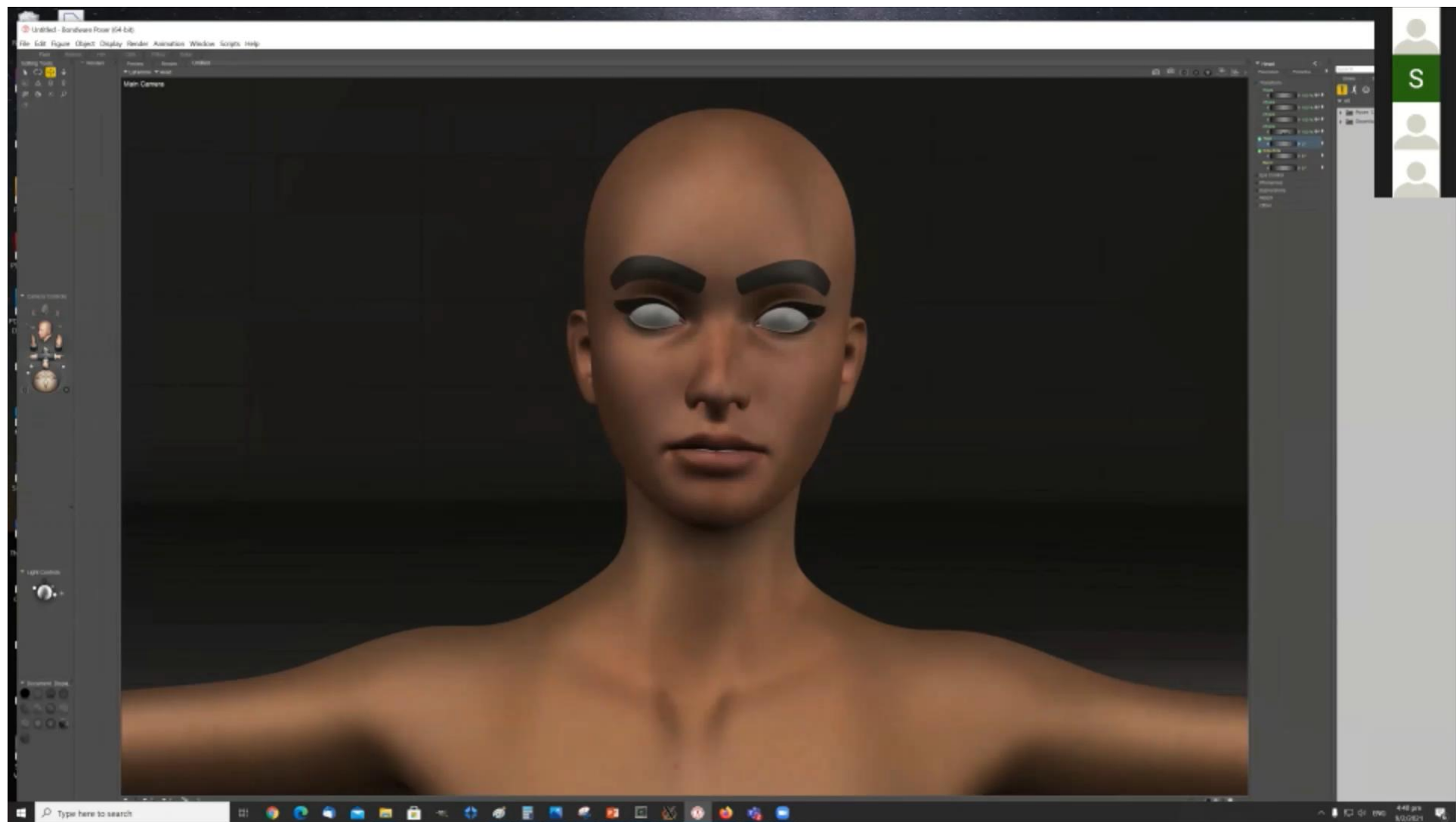
The one below is Z-X



# Problem: Gimbal Lock

- ❑ Two rotational axis of an object pointing in the same direction - 1DOF is lost
  - For example for rotation defined in the order of X-Y-Z
  - Gimbal lock occurs when rotating Y for 90 degrees.
  - X and Z axis get pointed down the same axis





01/17/14

# Free joint

- ❑ A ball joint
- ❑ 3 DOF
- ❑ Do not have to worry about gimbal lock

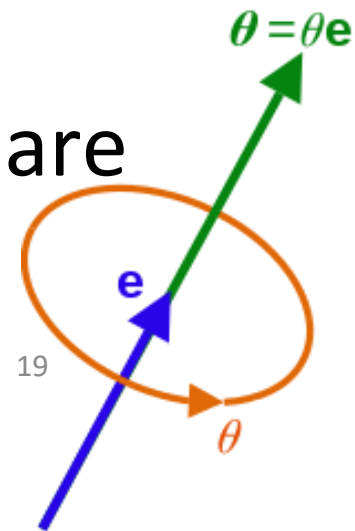


# Axis-Angle Representation

Euler's theorem: Any rotation or sequence of rotations of a rigid body in a three-dimensional space is equivalent to a pure rotation about a single fixed axis.

Axis-Angle representation: Represent a rotation by an axis of rotation  $\mathbf{e}$ , and the angle of rotation  $\theta$  about this axis

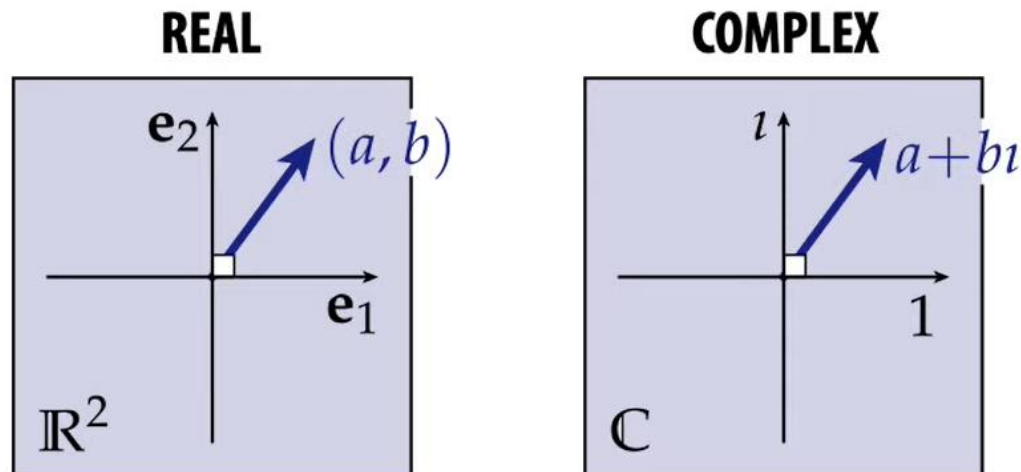
Simple, but operations such as compositions are not trivial.



# Complex numbers for rotation

Complex numbers can be applied as a representation for 2D geometry

Instead of representing the 2D plane by two real numbers, we can represent it by a pair of real and imaginary number



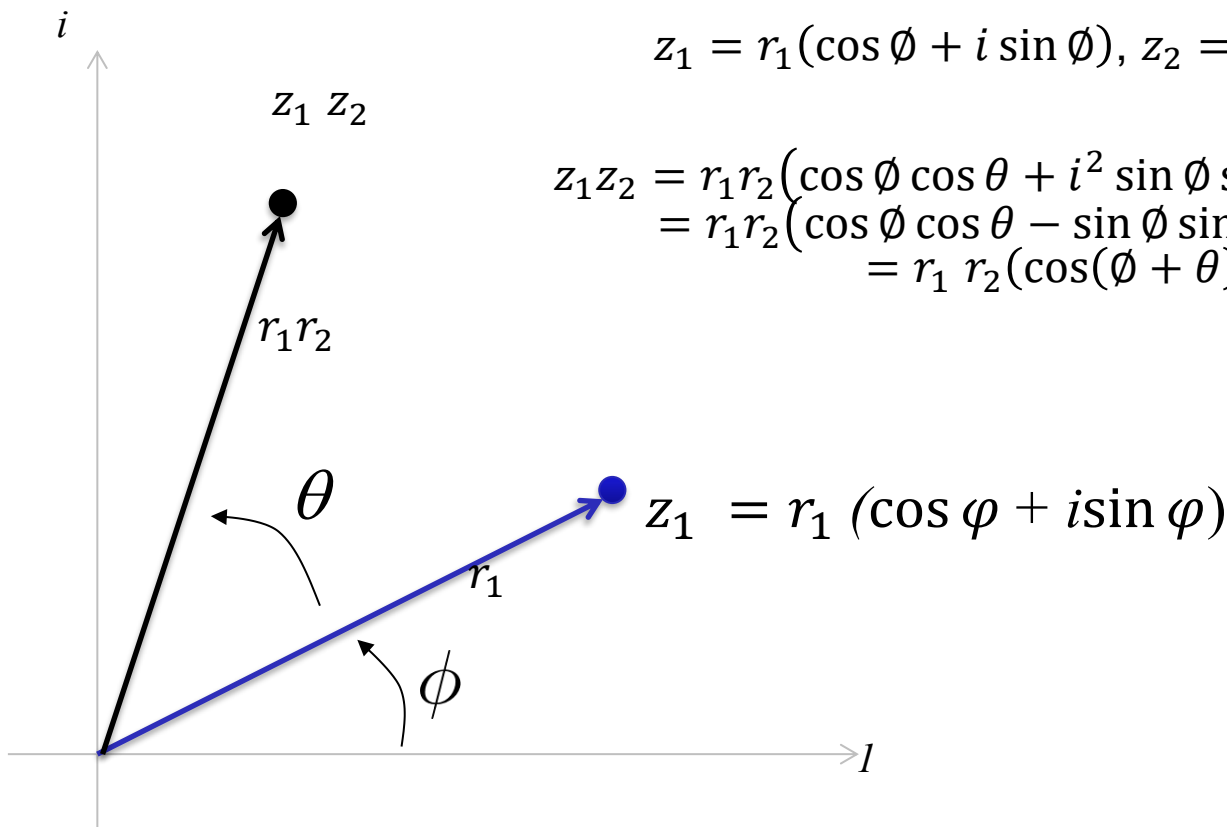
# Complex analysis

Let's represent the vector in the polar form

Let's multiply two complex numbers in this form

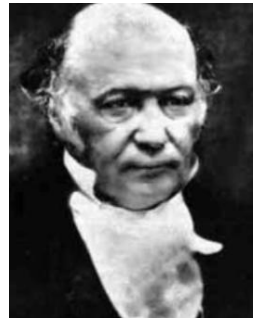
$$z_1 = r_1(\cos \phi + i \sin \phi), z_2 = r_2(\cos \theta + i \sin \theta)$$

$$\begin{aligned} z_1 z_2 &= r_1 r_2 (\cos \phi \cos \theta + i^2 \sin \phi \sin \theta + i(\cos \phi \sin \theta + \sin \phi \cos \theta)) \\ &= r_1 r_2 (\cos \phi \cos \theta - \sin \phi \sin \theta + i(\cos \phi \sin \theta + \sin \phi \cos \theta)) \\ &= r_1 r_2 (\cos(\phi + \theta) + i \sin(\phi + \theta)) \end{aligned}$$



# Quaternions

Mathematical object developed by Sir William Rowan Hamilton in 1843 as an extension to the complex numbers



General form of a quaternion:

$$\mathbf{q} = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$$

where  $\mathbf{i}, \mathbf{j}, \mathbf{k}$  are “complex” numbers such that

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$$

A **quaternion** can therefore be represented as a 4-dimensional vector

$$\mathbf{q} = (w, x, y, z)$$

# Quaternions

The  $xi + yj + zk$  part is similar to a 3D vector, so we may express a quaternion as

$$\mathbf{q} = (w, \mathbf{v})$$

scalar

3D vector

# Quaternion Operation: Multiplication

We can compute a product of quaternions

$$\begin{aligned} q_1 q_2 &= (w_1 + \mathbf{v}_1)(w_2 + \mathbf{v}_2) \\ &= w_1 w_2 + w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \mathbf{v}_2 \end{aligned}$$

$$\begin{aligned} \mathbf{v}_1 \mathbf{v}_2 &= (x_1 \mathbf{i} + y_1 \mathbf{j} + z_1 \mathbf{k})(x_2 \mathbf{i} + y_2 \mathbf{j} + z_2 \mathbf{k}) \\ &= -x_1 x_2 - y_1 y_2 - z_1 z_2 + (y_1 z_2 - y_2 z_1) \mathbf{i} + (z_1 x_2 - z_2 x_1) \mathbf{j} + (x_1 y_2 - x_2 y_1) \mathbf{k} \\ &= -\mathbf{v}_1 \cdot \mathbf{v}_2 + \mathbf{v}_1 \times \mathbf{v}_2 \end{aligned}$$

Altogether we have

$$q_1 q_2 = w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 + w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2$$

## Other operators:

Given a quaternion  $q = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k} = w + \mathbf{v}$

Complex conjugate:  $q^* = w - x\mathbf{i} - y\mathbf{j} - z\mathbf{k} = w - \mathbf{v}$

Quaternion norm:  $|q| = \sqrt{qq^*} =$   
 $\sqrt{w^2 + x^2 + y^2 + z^2}$

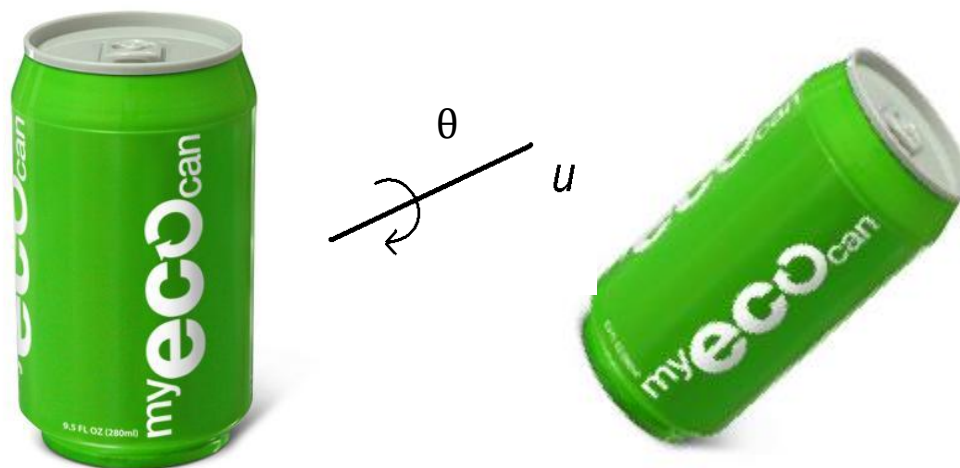
Inverse of quaternion :  $q^{-1} = \frac{q^*}{|q|^2} \quad qq^{-1} = 1$

# Theorem

A unit quaternion in the following form:

$$q = w + \mathbf{q} = \cos \frac{\theta}{2} + \mathbf{u} \sin \frac{\theta}{2}$$

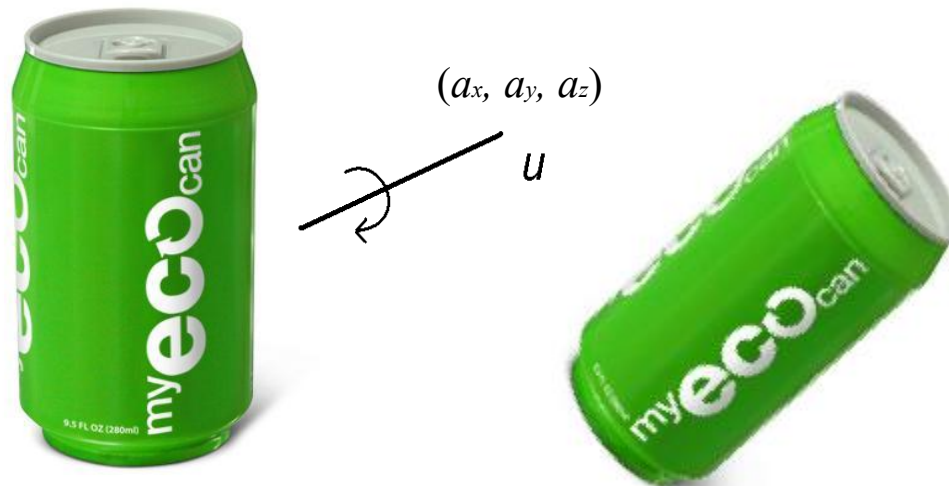
represents a rotation of  $\theta$  around  $\mathbf{u}$



# Theorem

Applying a rotation to a vector  $v$  can be computed by

$$L_q(v) = qvq^*$$



# A vector in quaternion format

A vector is represented as a quaternion by setting the scalar part 0:

$$\mathbf{q}_{\mathbf{u}} = (0, \mathbf{u})$$

# Quaternion Composition

Let  $q_1$  and  $q_2$  be two unit quaternions representing two rotations.

The composition of first rotation by  $q_1$  and then rotation by  $q_2$  is given by the multiplication of  $q_2$  and  $q_1$ :

$$q_2 q_1 = (w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_2 \times \mathbf{v}_1)$$

  
Order matters!

  
Vector dot product

  
Vector cross product

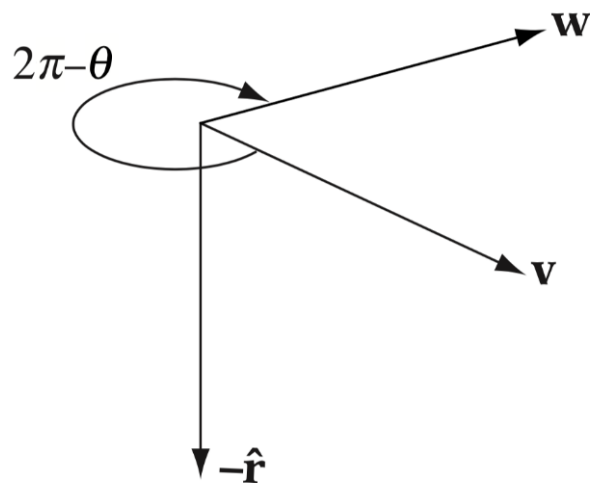
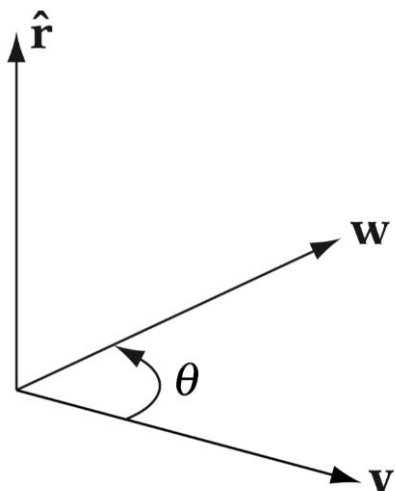
# Quaternion Negation

$$\mathbf{q} = \left( \cos \frac{\theta}{2}, \sin \frac{\theta}{2} \mathbf{r} \right)$$

$$-\mathbf{q} = \left( -\cos \frac{\theta}{2}, -\sin \frac{\theta}{2} \mathbf{r} \right)$$

$$= \left( \cos \frac{2\pi + \theta}{2}, \sin \frac{2\pi + \theta}{2} \mathbf{r} \right)$$

$$= \left( \cos \frac{2\pi - \theta}{2}, \sin \frac{2\pi - \theta}{2} (-\mathbf{r}) \right)$$



# Issues with quaternions

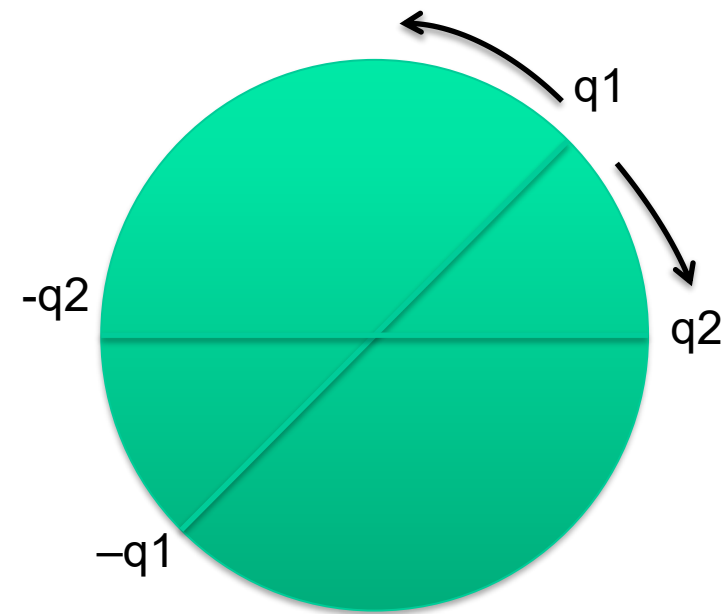
The mapping from space of rotation (called  $SO(3)$ ) to unit quaternions is 1 to 2

Because  $q$  and  $-q$  represent the same rotation.

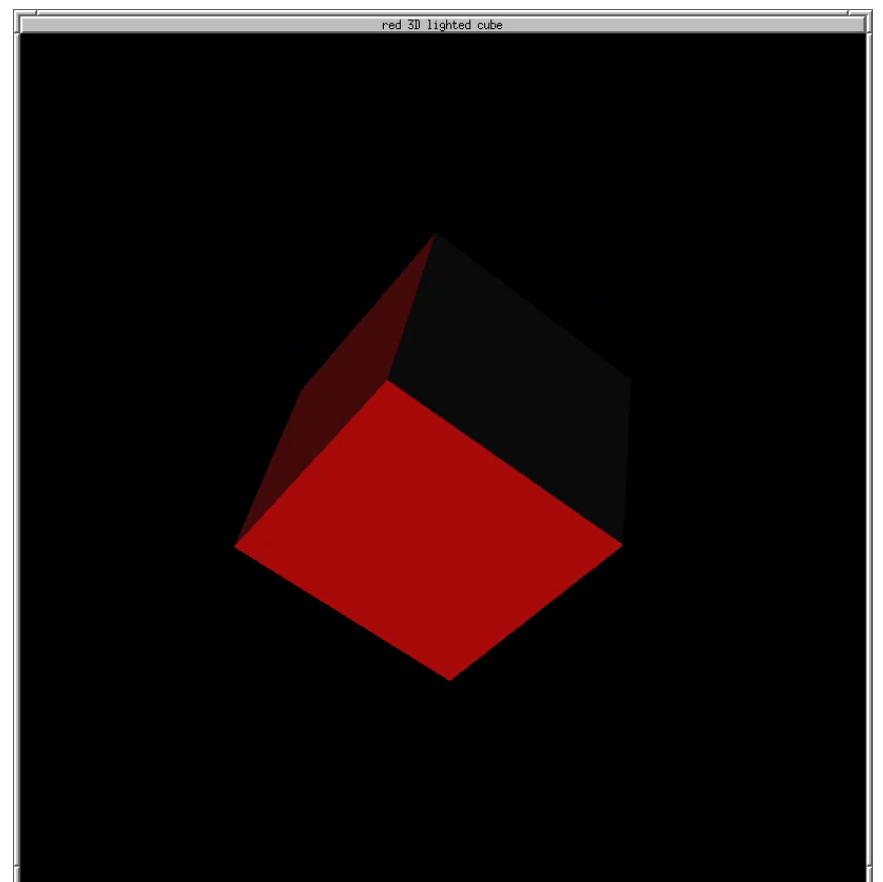
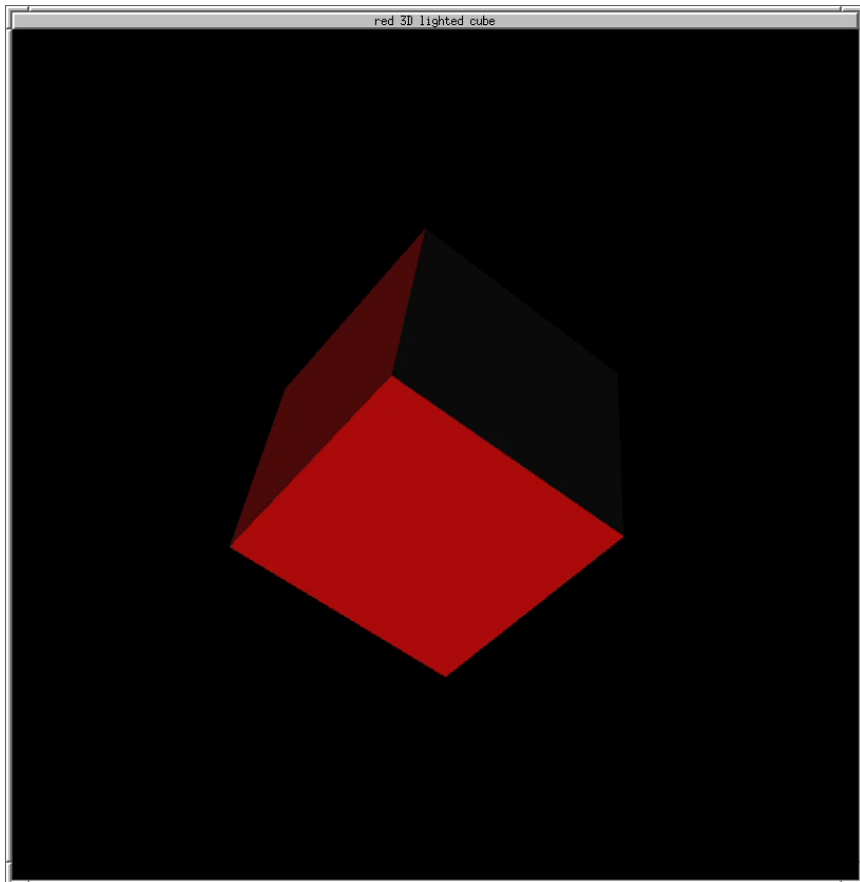
When starting from a orientation  $q_1$  towards  $q_2$  or from  $q_1$  towards  $-q_2$ , the rotation will go the other way around.

Solution: Compute the dot product with  $q_1$  and  $q_2$  or  $-q_2$

Interpolate with the one whose dot product is positive



# Negating q2

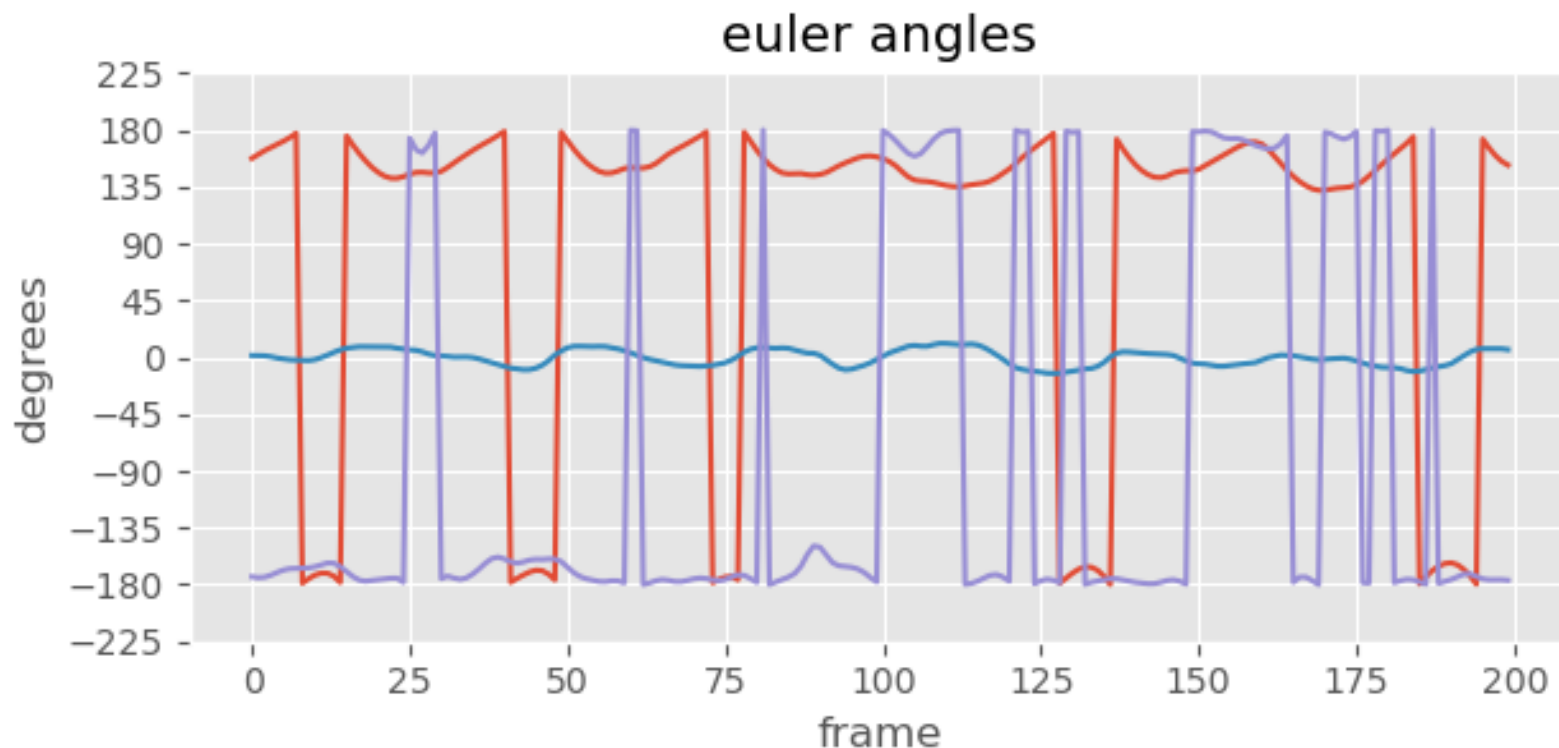


# Unrolling

- Rotation representations
  - 3x3 Rotation matrix
  - Euler angles
  - Axis-angles
  - Quaternions
- Some of these representations have
  - Multiple values for the same orientation
  - Which one has this issue?

# Rotation Trajectory in Euler Angles

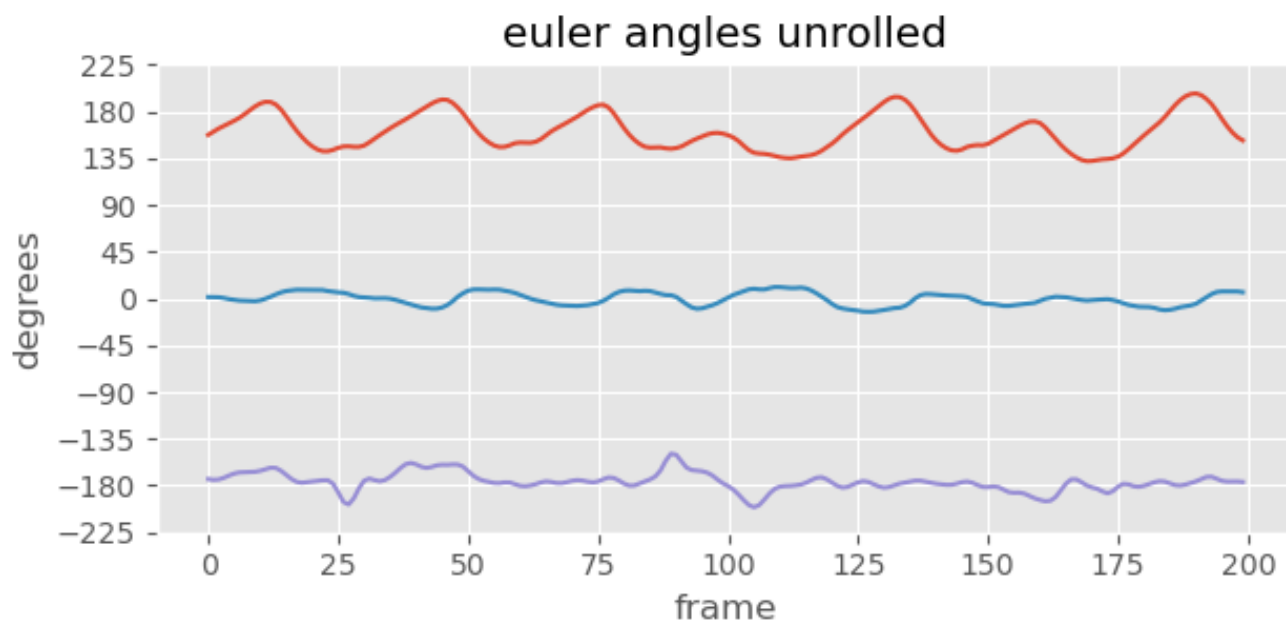
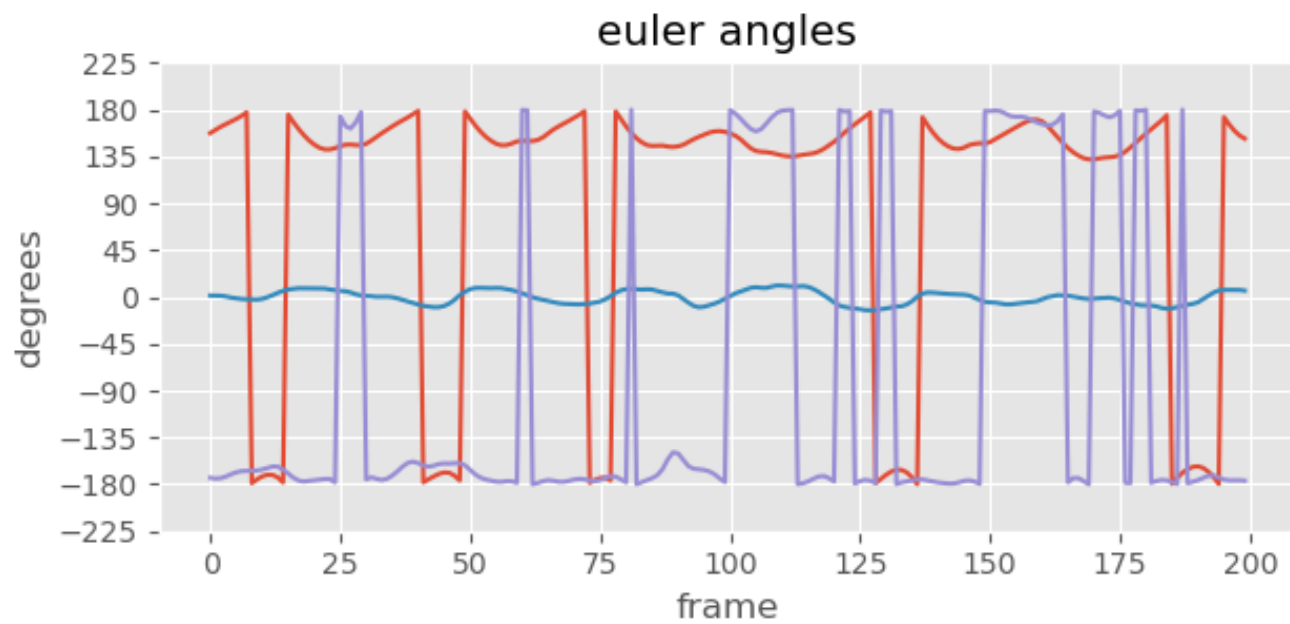
Here is a trajectory of some 3DOF joint



Why does this happen?

# Unrolling the Euler Angles

- Compute the difference of consecutive frames
- Normalize them so that their range is within -180 to 180 degrees
- Add them back to the previous frame angle

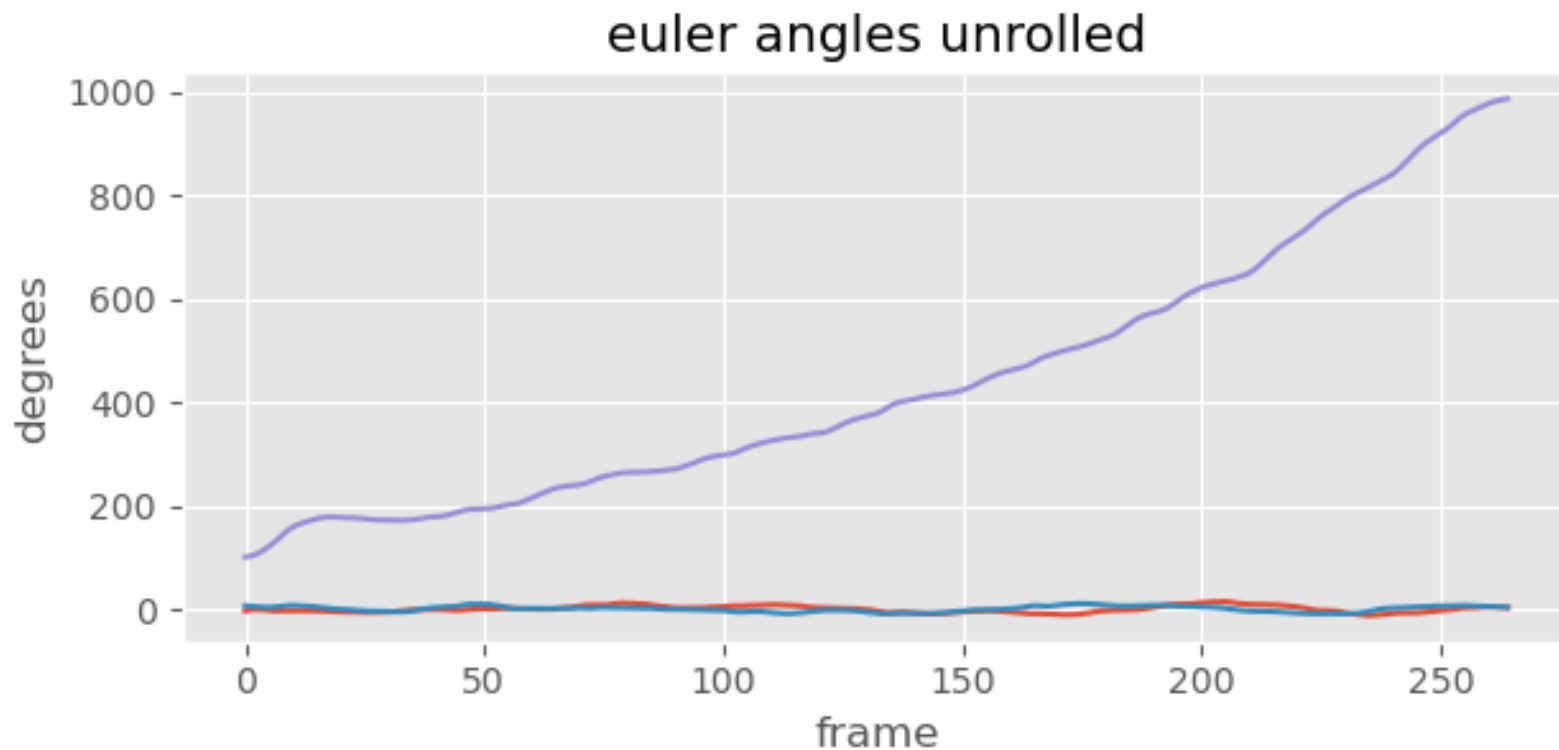


# Unrolling the Euler Angles (2)

- What if we have a wheel of a buggy that rotates over and over?
- What would the Euler angles look like?

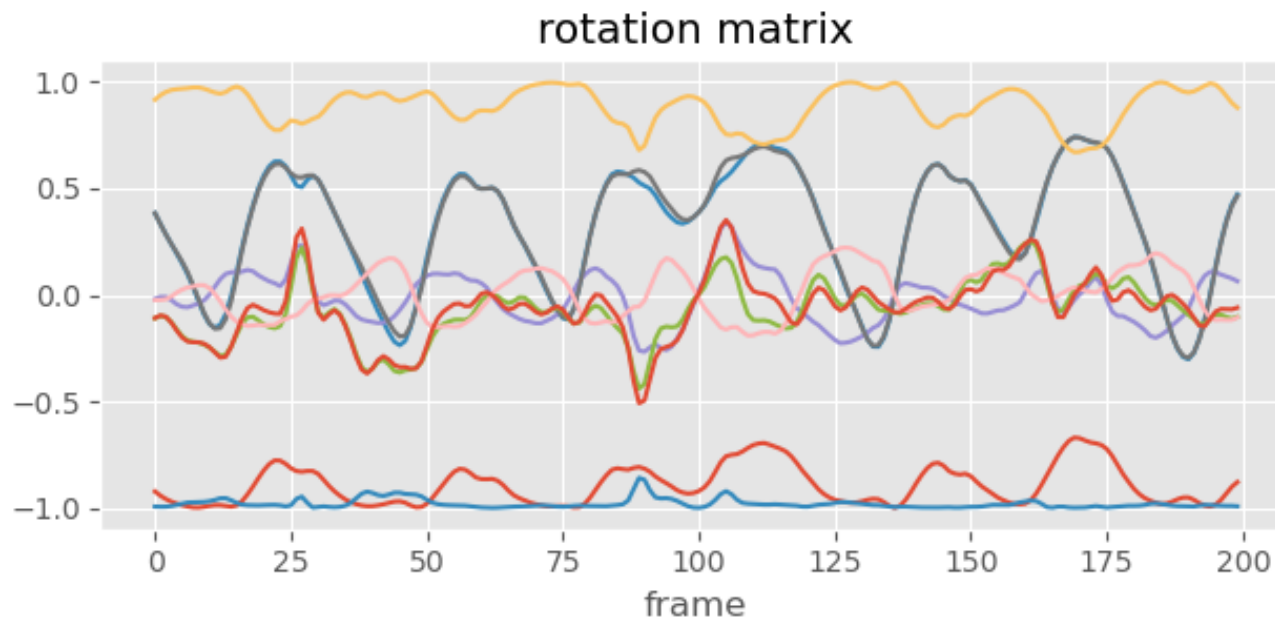


# Unrolling the Euler Angles (2)



# What about Rotation Matrix?

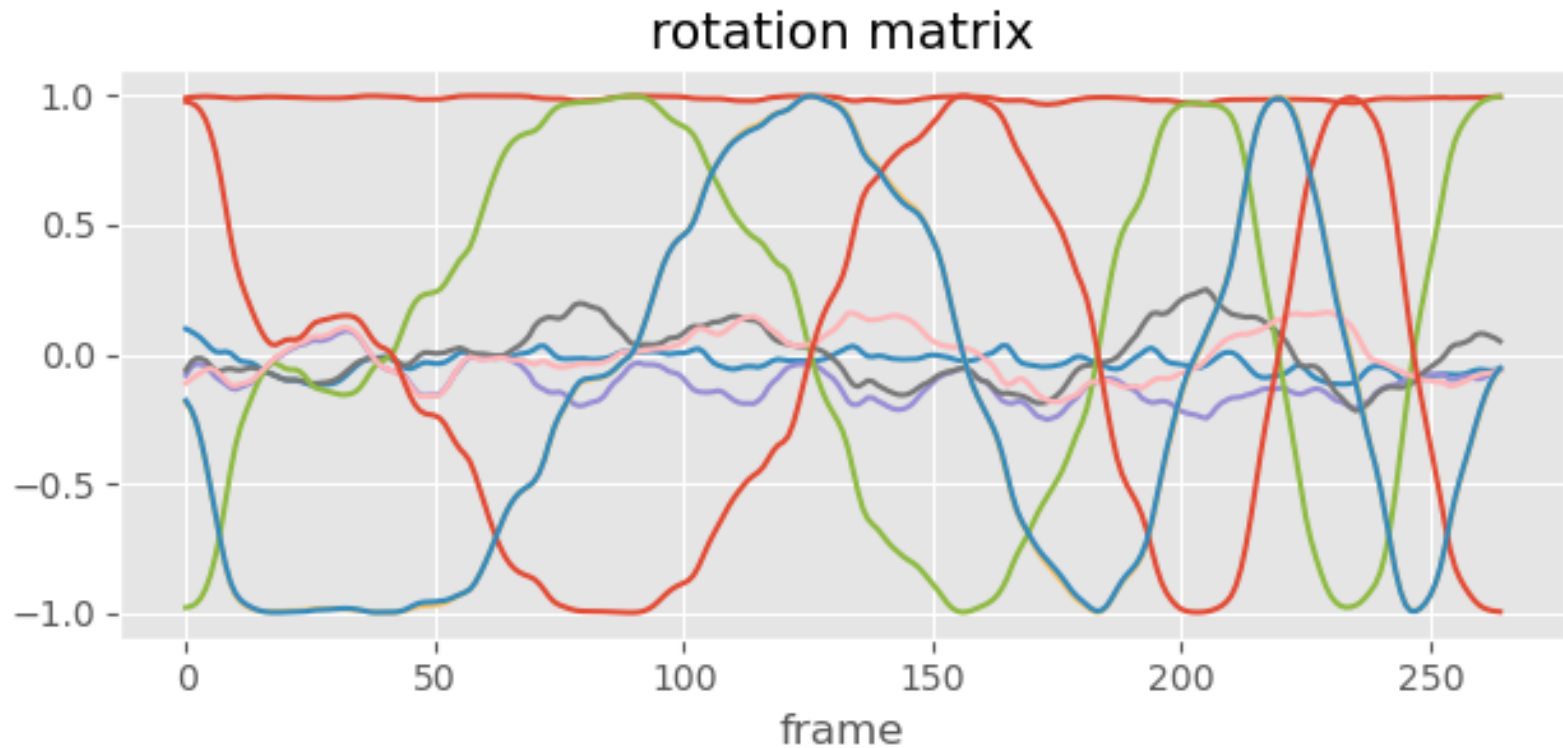
Can there be any discontinuities?



Why not?

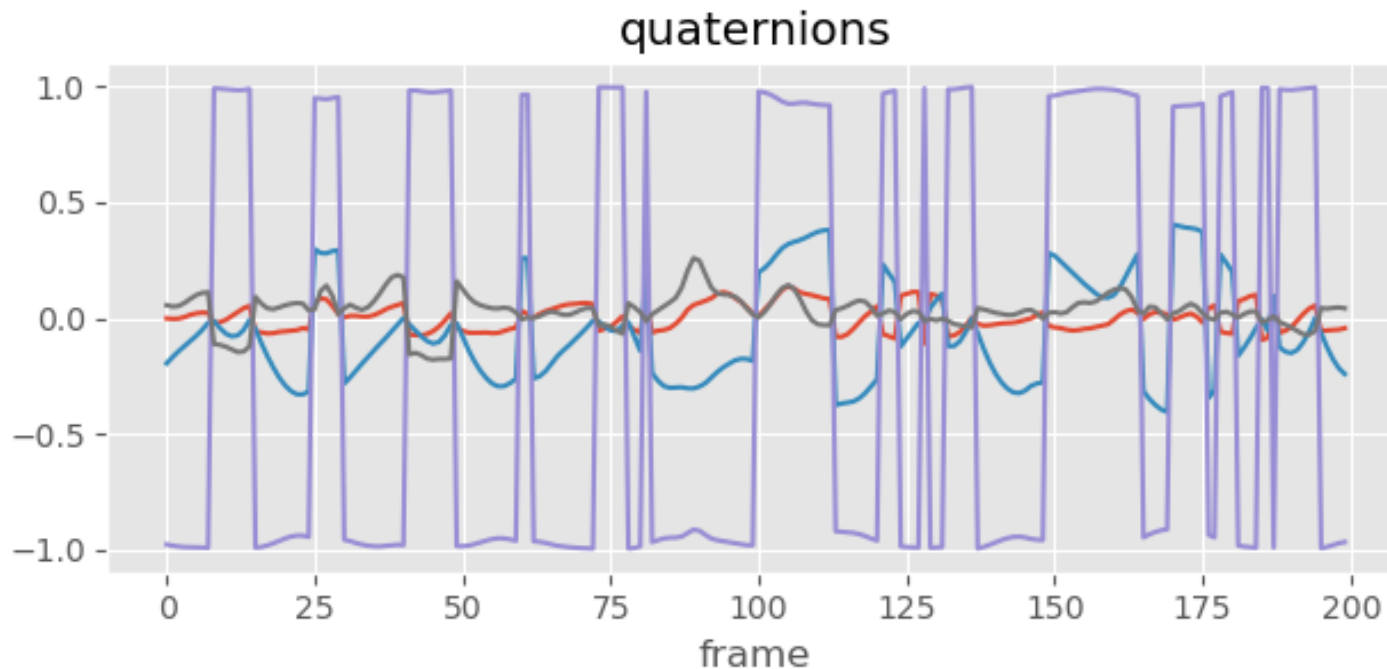
What if for the buggy wheel example?

# Rotation matrix for the Buggy Wheel



# Unrolling the Quaternions

- Can there be discontinuities with quaternions?
- Why?



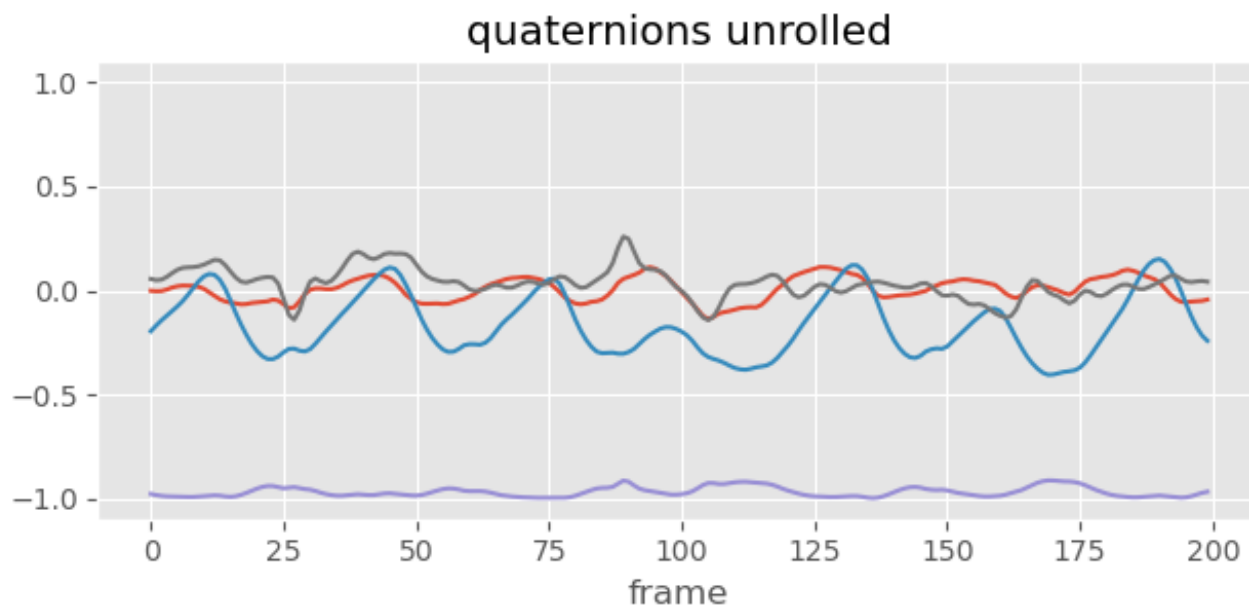
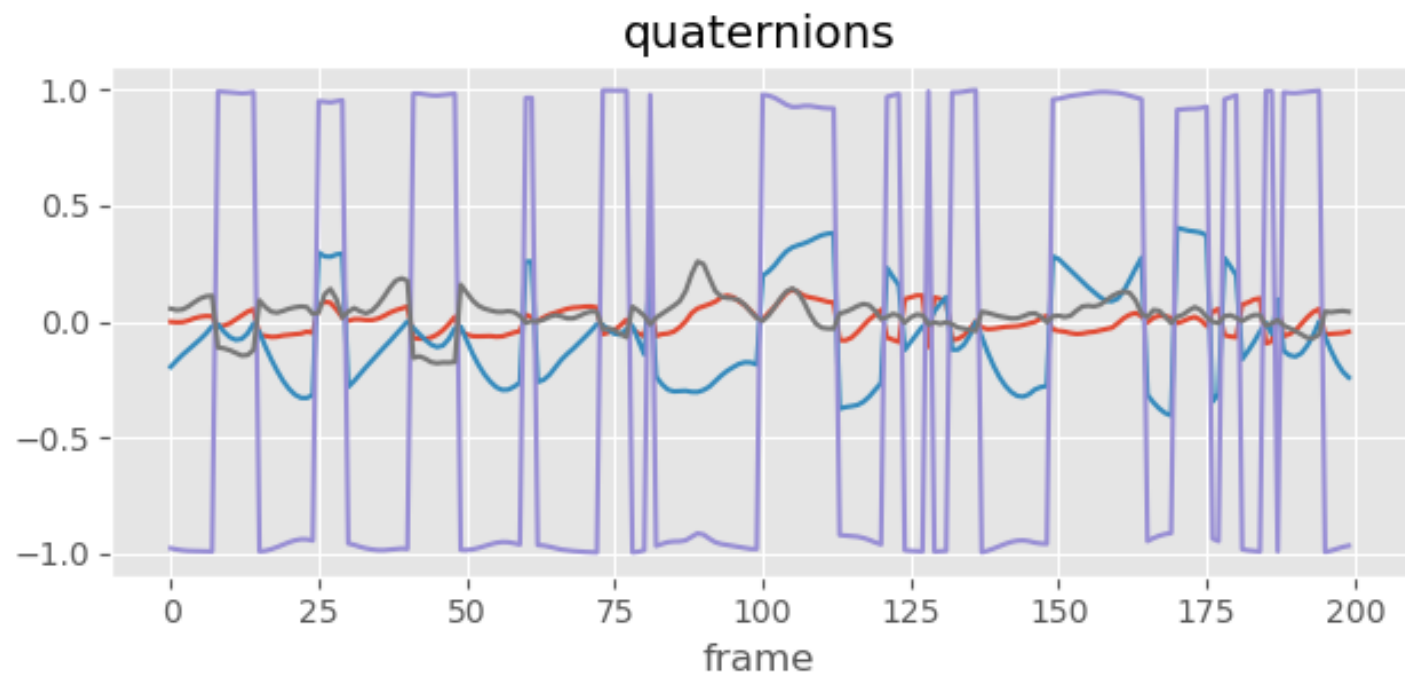
# Unrolling the Quaternions

We can compute the dot product between the two consecutive frames

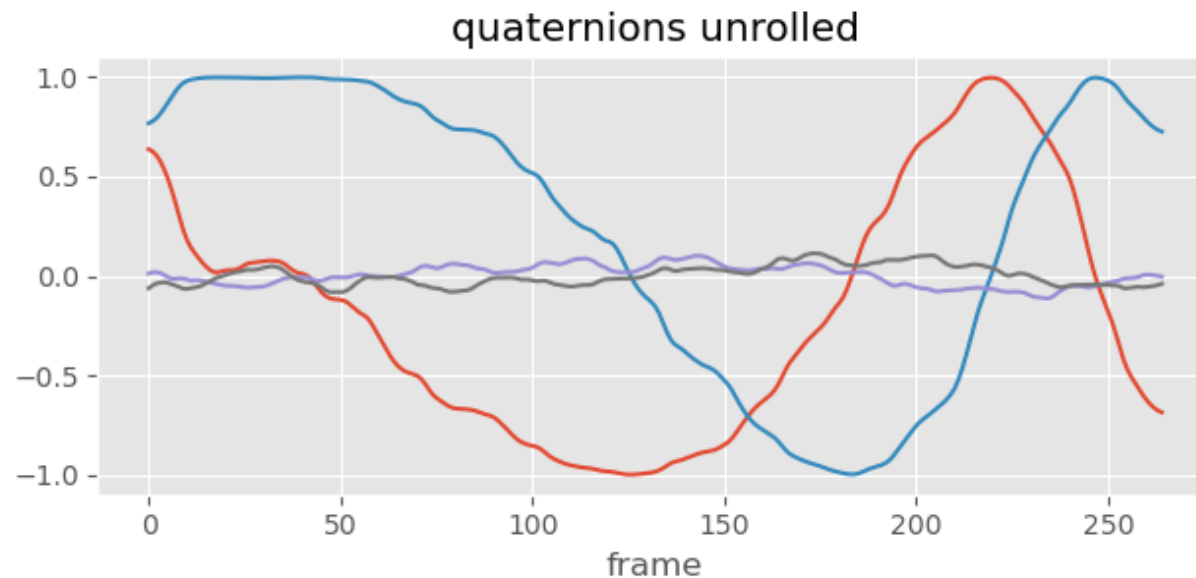
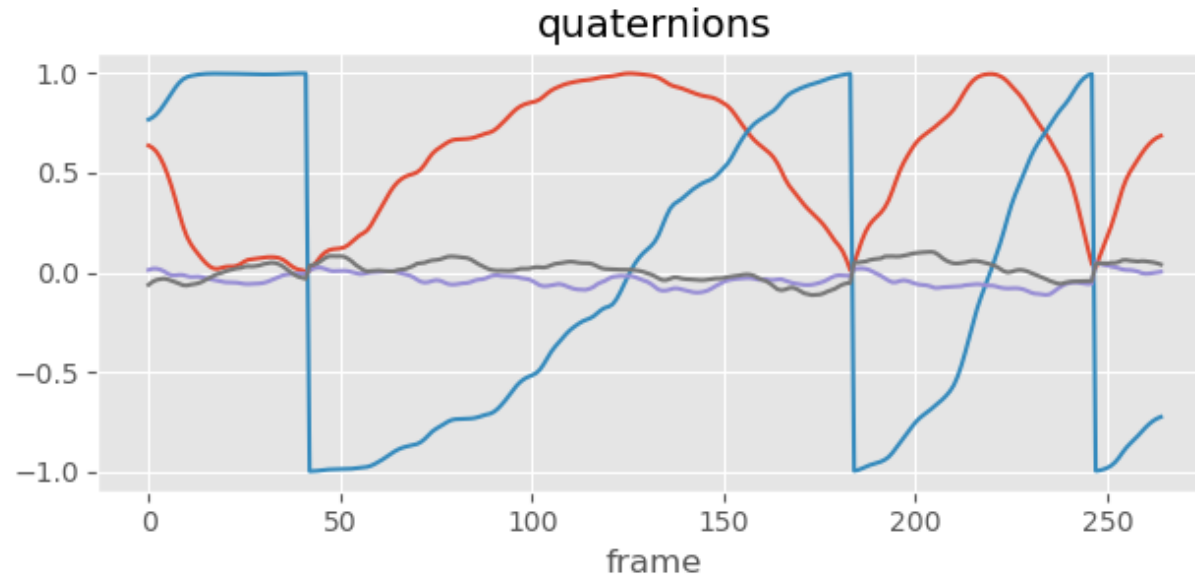
$$\mathbf{q}_i \cdot \mathbf{q}_{i+1}$$

If the value is negative, we negate the next quaternion

$$\mathbf{q}_{i+1} \leftarrow -\mathbf{q}_{i+1}$$



# Unrolling Quaternions with the Buggy Example



# Summary

- Various representations for rotation
  - 3x3 Rotation matrix
  - Euler angles
  - Axis-angles
  - Quaternions

# Generalized Coordinates

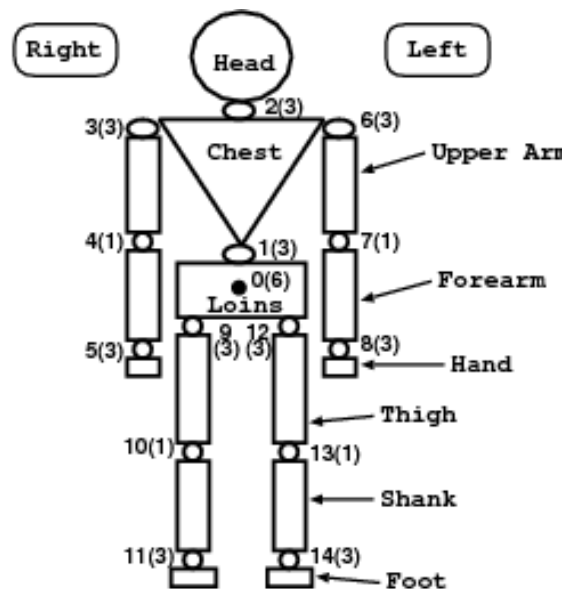
A vector to specify the posture of the body

$$q = (q_1, q_2, q_3, q_4, q_5, q_6, q_7, \dots, q_n)$$

Usually, the first three numbers: *location of the root*

The next three numbers: *orientation of root*

The rest: *the joint angles of the body*



# Overview

## ☐ Character Animation

### ■ Posture representation

#### ■ Hierarchical structure of the body

#### ■ Joint types

☐ Translational, hinge, universal, gimbal, free

#### ■ Euler angles

☐ Gimbal lock

#### ■ Quaternions

## ■ **Creating the animation (Keyframe animation)**

### ■ Interpolation

### ■ Inverse Kinematics

#### ■ Analytical

#### ■ CCD

#### ■ Pseudo Inverse

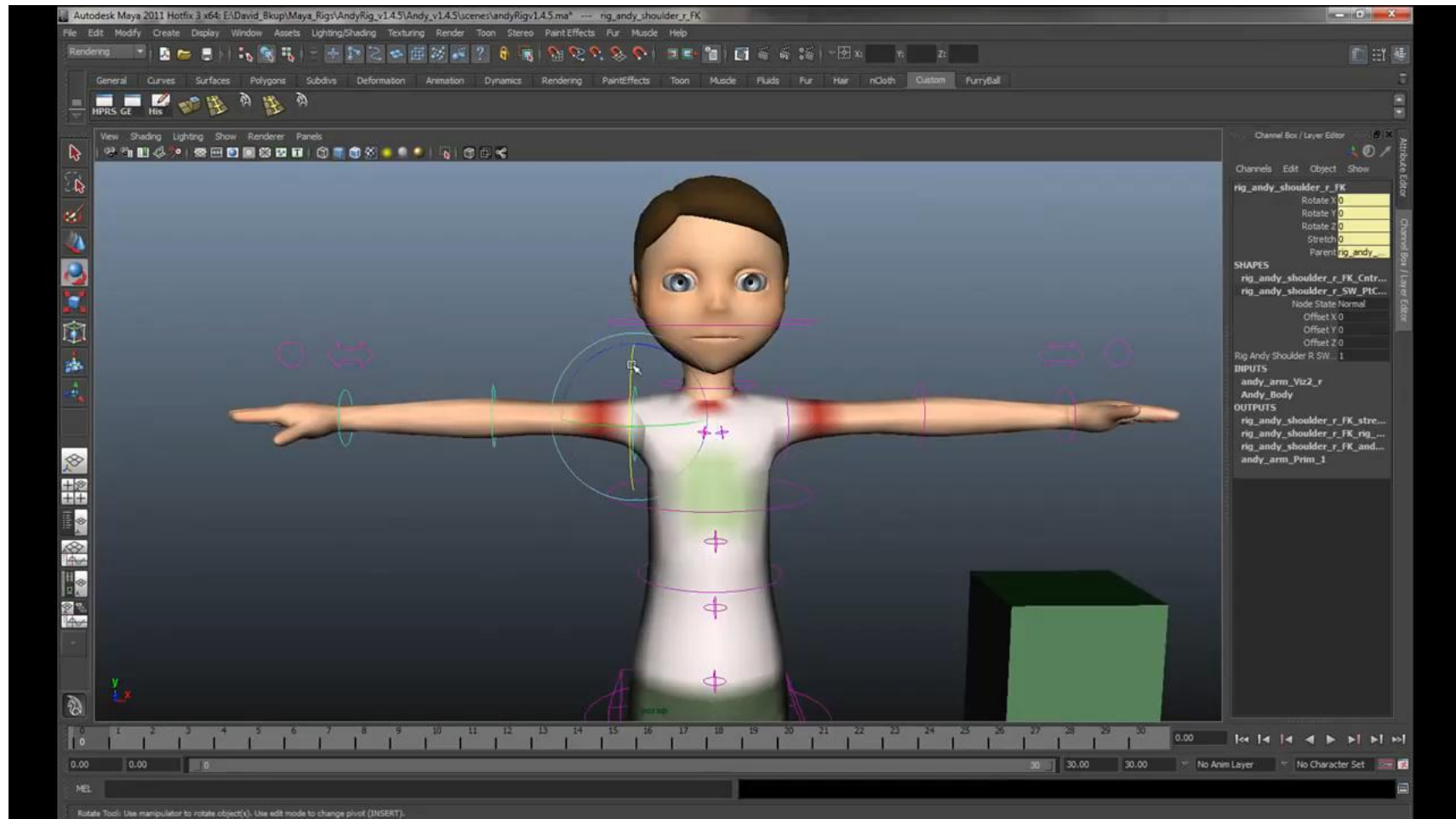
# Keyframe Animation

- ❑ The keyframe postures are designed by the animator
- ❑ The inbetween motion is created by interpolation



# Keyframe Design

- Each postures are created by the user interface
  - Forward Kinematics (FK, Virtual Track Ball)
    - The user clicks the segment and rotates it around the joint origin
    - The movement of the mouse is mapped to the rotation of the joint
    - Example: <https://www.youtube.com/watch?v=pGuR4xyi7MQ>
  - Inverse Kinematics (IK)
    - The user clicks the segment and drags it in the 3D coordinate
    - The motion of the mouse is mapped to the translation in 3D coordinate
    - The movement of the segment is achieved by moving each joint of the body
    - Example: <http://www.youtube.com/watch?v=--5hWniftyI>
    - Interactive Demo: <http://www.starke-consult.de/BioIK-Demo/index.html>



01/17/14

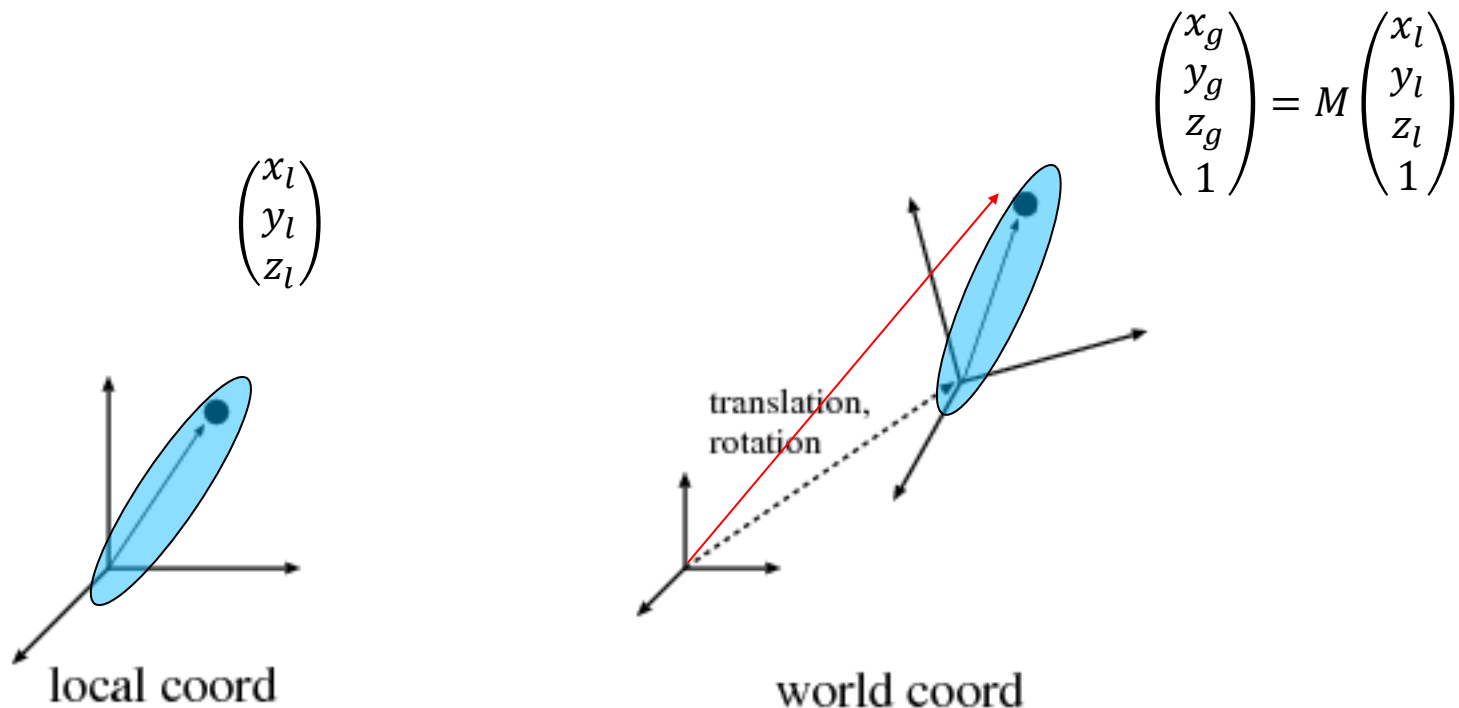
# Forward Kinematics

Computing the position of the joints/points on the body, given the body center position and joint angles

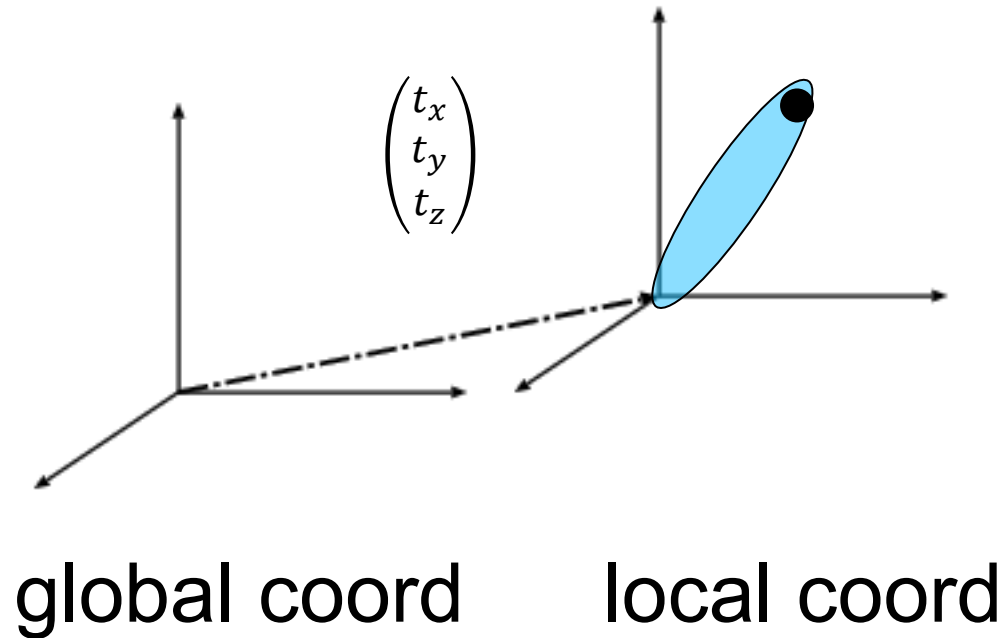


# Background

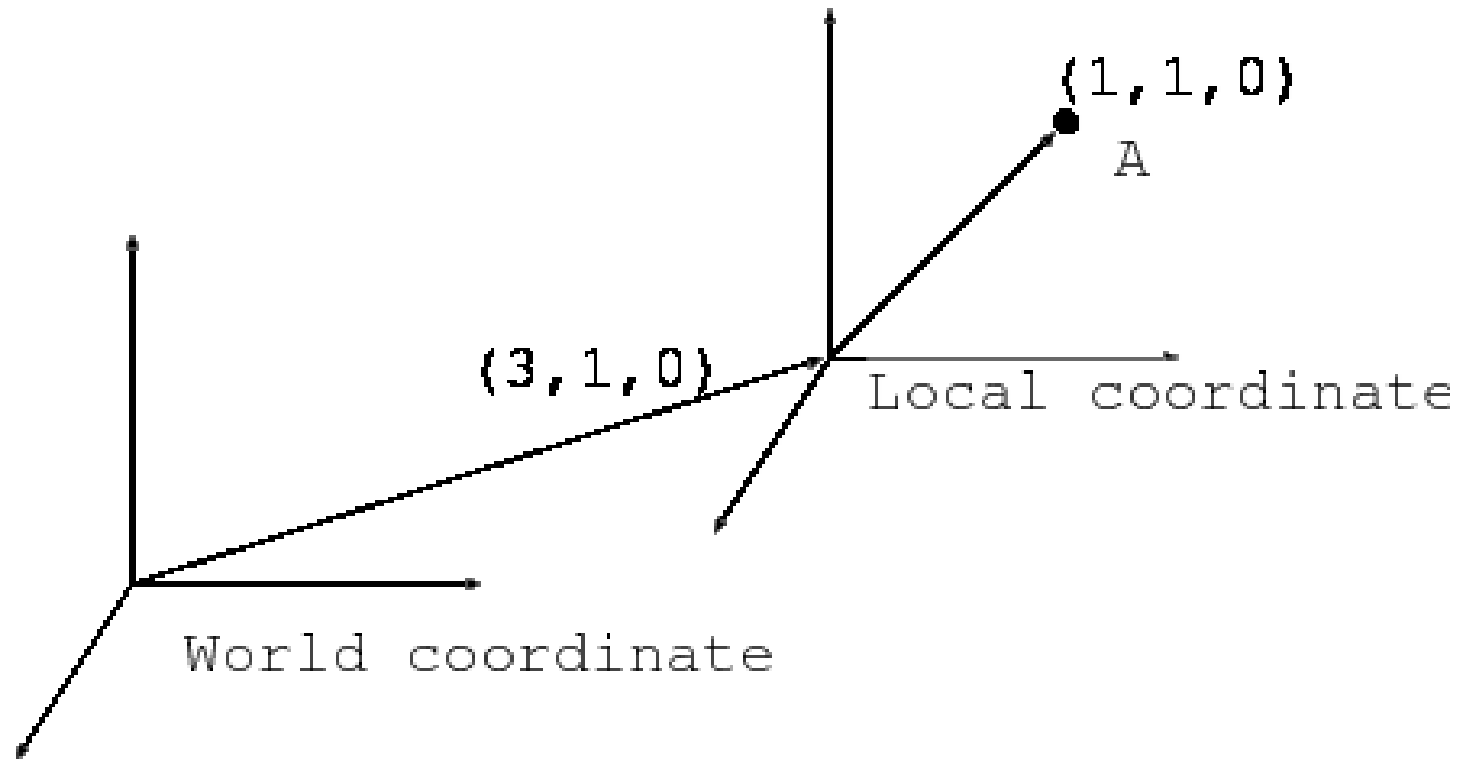
- A homogeneous transformation matrix  $M$  (4x4) is defined per bone
- It represents the pose of the bone
- It converts local coordinates to world coordinates



# Translation matrix



$$M = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



- What is the local-to-world transformation matrix?
- What is the position of point A in the world coordinate system?

$$M = \begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \\ 0 \\ 1 \end{bmatrix}$$

- So the location of the point in the world is (4,2,0)

# Rotation matrix



$$Rot\ x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

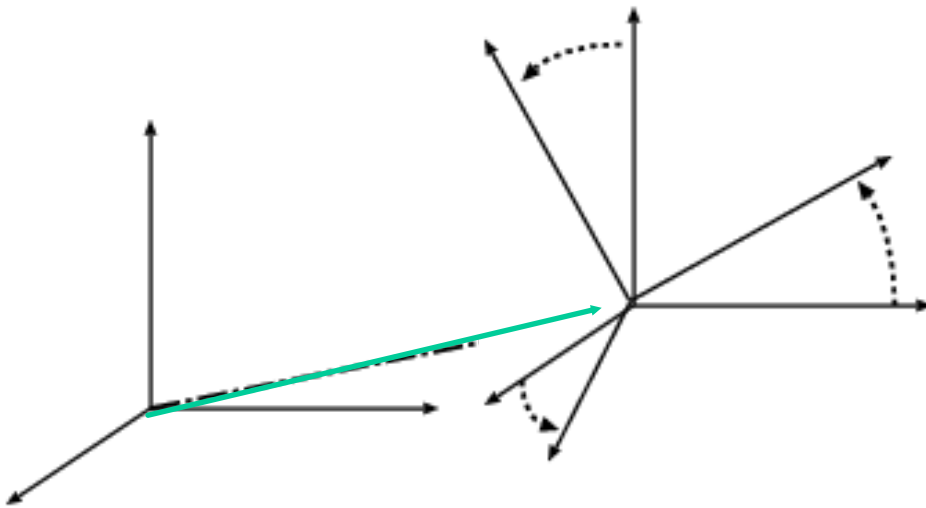
$$Rot\ y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$Rot\ z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- We assume we move together with the local coordinate system while transforming.
- Then, for Euler angles in the order of x, y, z, the matrix can be computed by

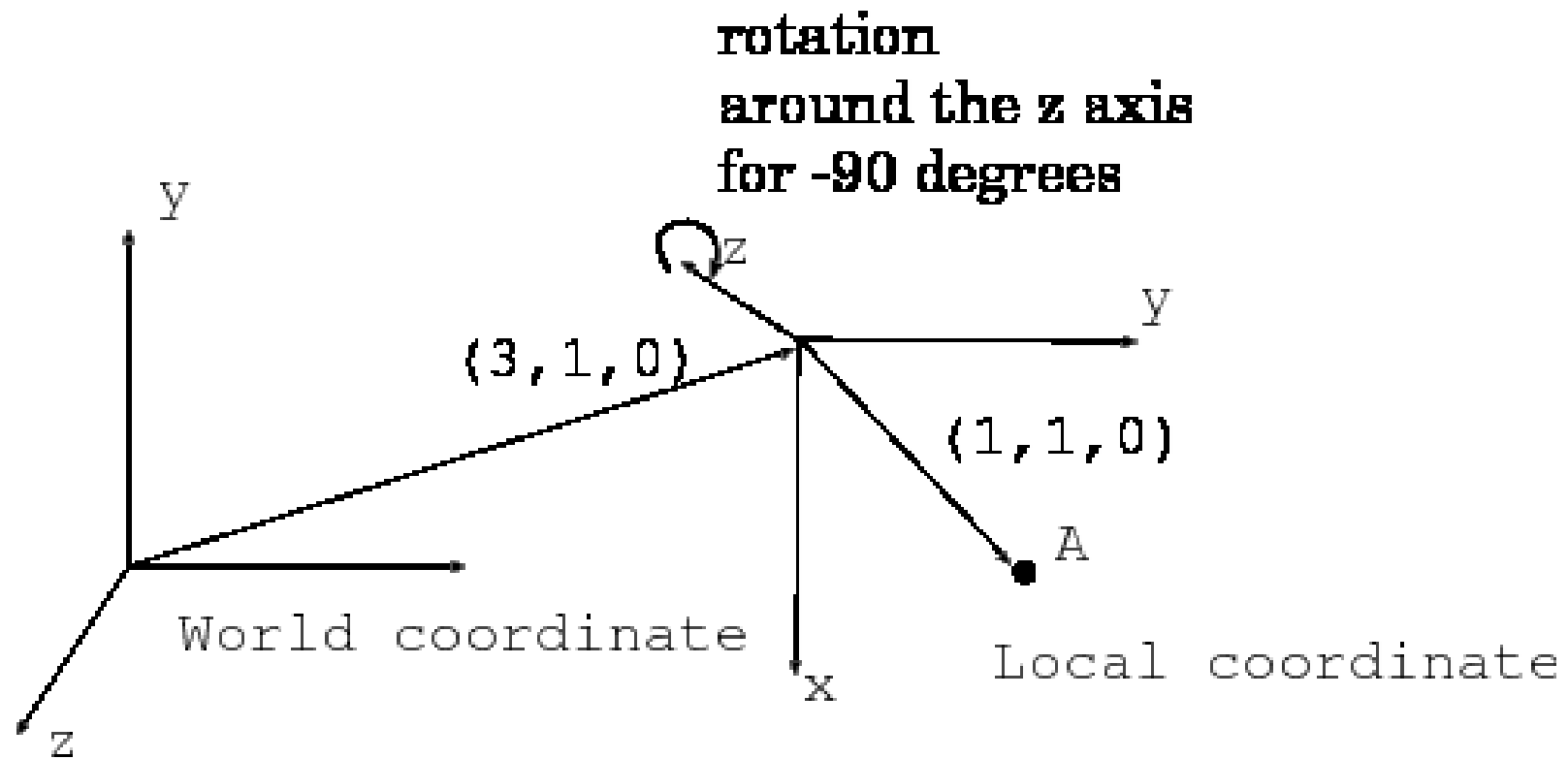
$$\begin{bmatrix} \mathbf{R}_{\alpha\beta\gamma} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} = Rot\ x(\alpha) \cdot Rot\ y(\beta) \cdot Rot\ z(\gamma)$$

# Translation & Rotation matrix



$$M = \begin{bmatrix} \mathbf{R}_{\alpha\beta\gamma} & \begin{matrix} t_x \\ t_y \\ t_z \end{matrix} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- First translating the local coordinate to  $t_x, t_y, t_z$
- And then rotating the local coordinate system with the Euler angles in the order of  $x, y, z$ ,
- The matrix can be computed by  $\text{Trans}(t_x, t_y, t_z) \cdot \text{Rot } x(\alpha) \cdot \text{Rot } y(\beta) \cdot \text{Rot } z(\gamma)$



- What is the local-to-world transformation matrix?
- What is the position of point  $A$  in the world coordinate system?

$$Rot_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

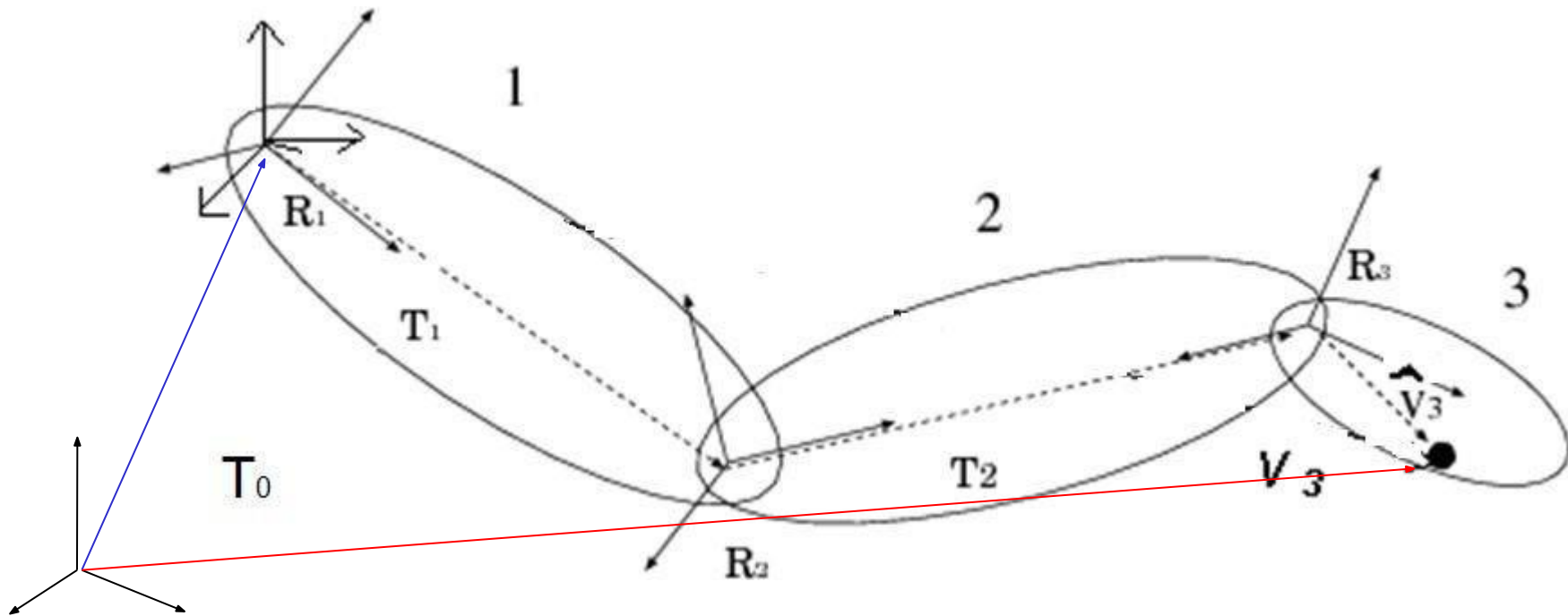
So rotating around the z axis for 90 degrees is

$$Rot_z(-\frac{\pi}{2}) = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 & 3 \\ -1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

- So the location of the point in the world is (4,0,0)

# A local-to-global transformation for a hand



$$\mathbf{v}_3 = \mathbf{T}_0 \mathbf{R}_1 \mathbf{T}_1 \mathbf{R}_2 \mathbf{T}_2 \mathbf{R}_3 \hat{\mathbf{v}}_3 = \mathbf{M}_3 \hat{\mathbf{v}}_3$$

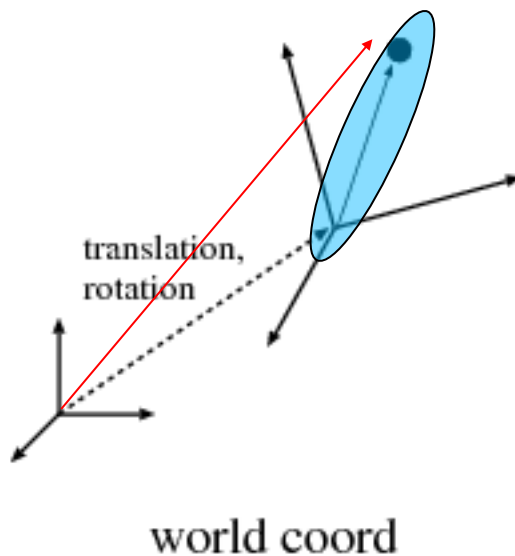
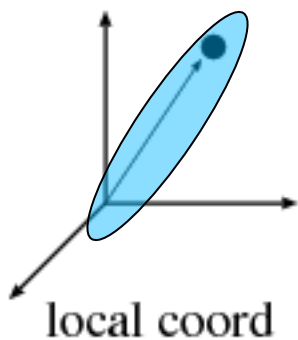
- A body composed of three segments
- A vertex  $\mathbf{v}_3$  defined in a local coordinate system of segment 3
- Its global position  $\mathbf{v}$  is computed by the above equation

# Forward Kinematics

- We use this kind of transformations a lot for computing the positions on the body after transformation
- We also use this a lot for skinning later

$$\begin{pmatrix} x_g \\ y_g \\ z_g \\ 1 \end{pmatrix} = M \begin{pmatrix} x_l \\ y_l \\ z_l \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x_l \\ y_l \\ z_l \end{pmatrix}$$



# Producing Keyframe Animation

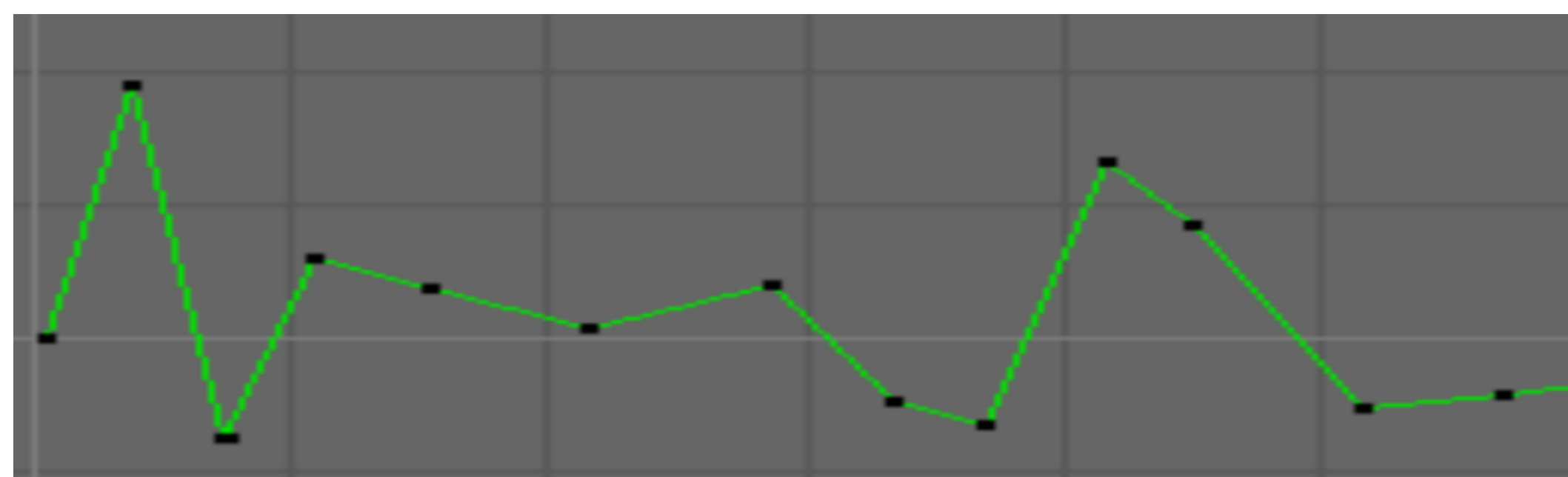
- ❑ Many software for keyframe animation
  - ❑ Blender, Maya, MotionBuilder



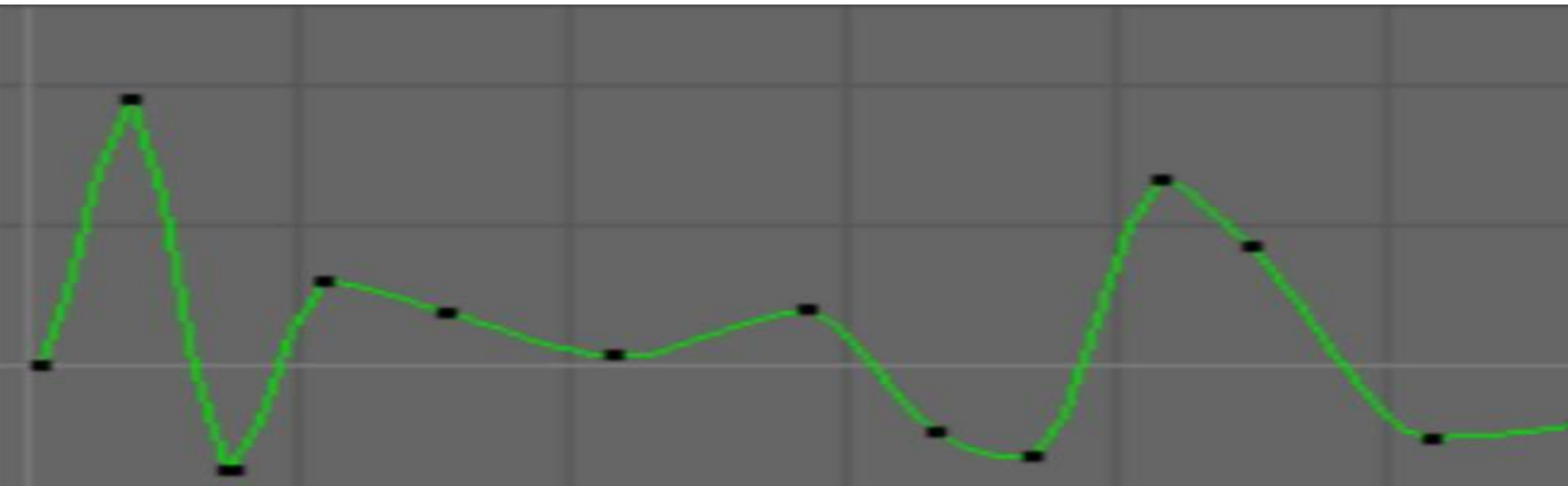
# Keyframe Animation

- ❑ The inbetween motion is created by interpolation
- ❑ How do we do we interpolate?
  - ❑ Position
  - ❑ Rotation
- ❑ Different types of interpolation
  - ❑ Linear
  - ❑ Cubic





Linear interpolation



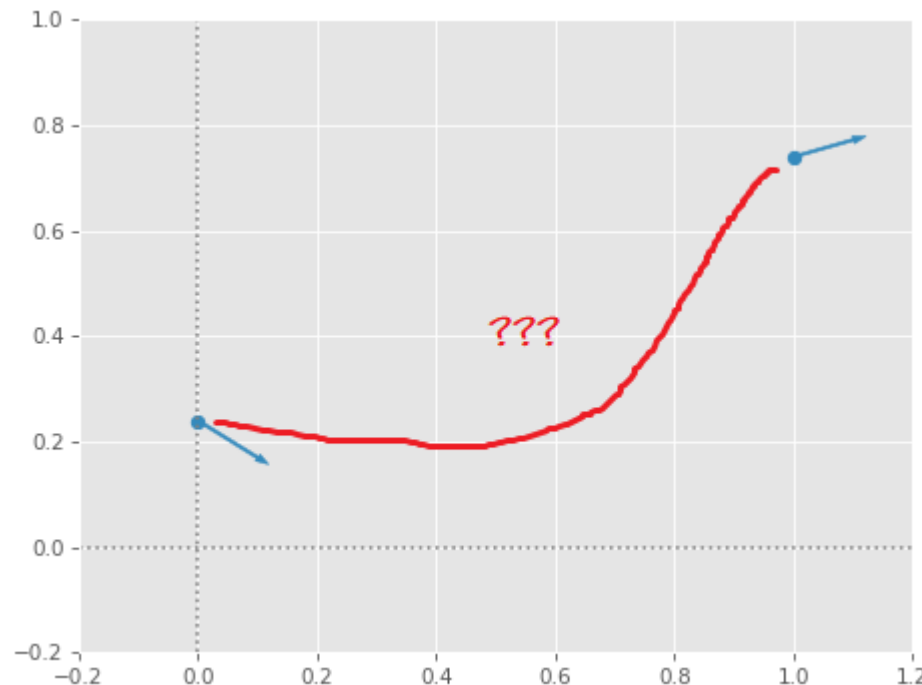
Cubic

# Hermite Curve

We have two points that we want to interpolate.

Also, we are given their gradients

How can we compute a cubic polynomial that interpolates?



# Hermite Curve

We fit a cubic polynomial function

$$\begin{aligned}f(x) &= a x^3 + b x^2 + c x + d \\f'(x) &= 3 a x^2 + 2 b x + c\end{aligned}$$

With boundary conditions

$$\begin{aligned}f(0) &= p_0 \\f(1) &= p_1 \\f'(0) &= v_0 \\f'(1) &= v_1\end{aligned}$$

# Hermite Curve

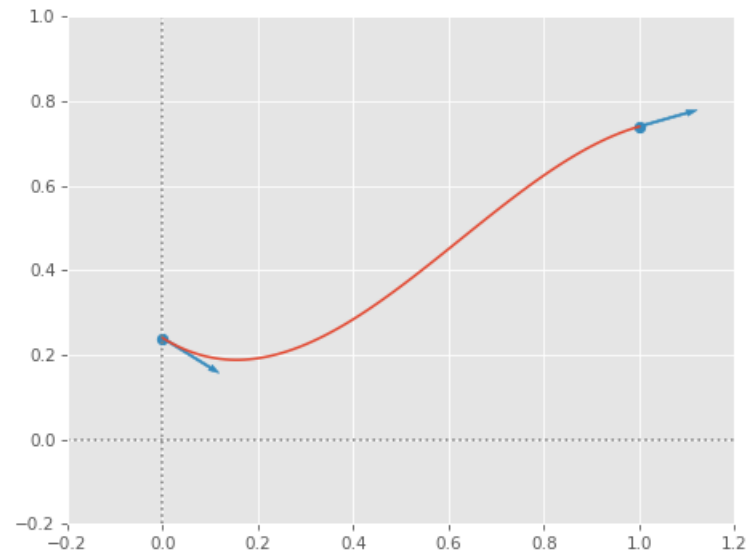
Solving for a,b,c, we get

$$f(x) = (v_0 + 2 p_0 + v_1 - 2 p_1) x^3 + (3 p_1 - 2 v_0 - 3 p_0 - v_1) x^2 + v_0 x + p_0$$

Or

$$f(x) = p_0 (2 x^3 - 3 x^2 + 1) + p_1 (3 x^2 - 2 x^3) + v_0 (x^3 - 2 x^2 + x) + v_1 (x^3 - x^2)$$

# Hermite Curve



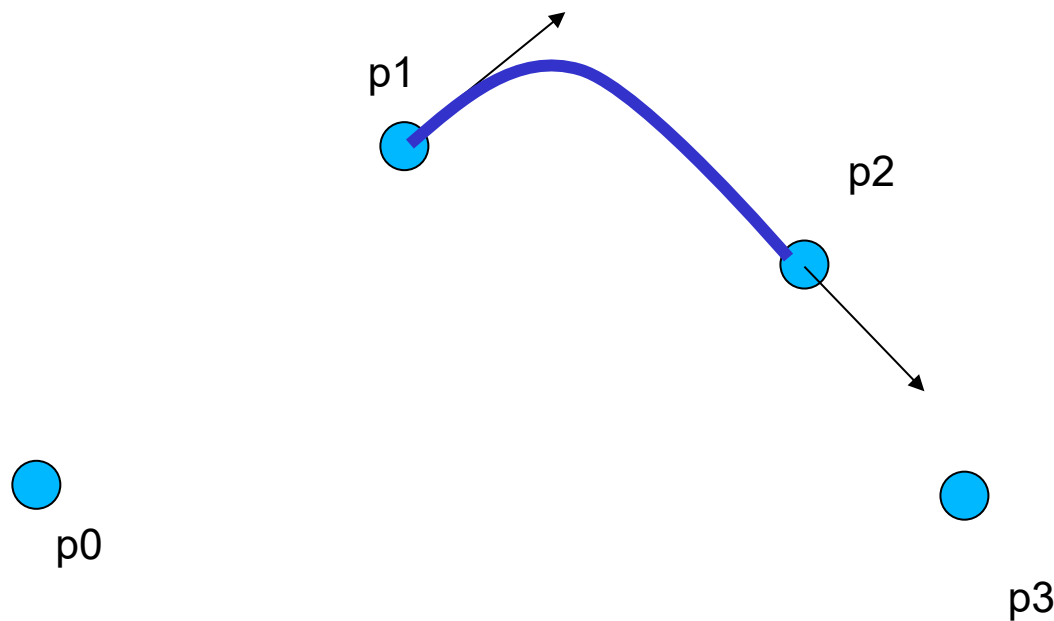
```
vec3 hermite_basic(float x, vec3 p0, vec3 p1, vec3 v0, vec3 v1)
{
    float w0 = 2*x*x*x - 3*x*x + 1;
    float w1 = 3*x*x - 2*x*x*x;
    float w2 = x*x*x - 2*x*x + x;
    float w3 = x*x*x - x*x;

    return w0*p0 + w1*p1 + w2*v0 + w3*v1;
}
```

# Catmull-Rom Spline

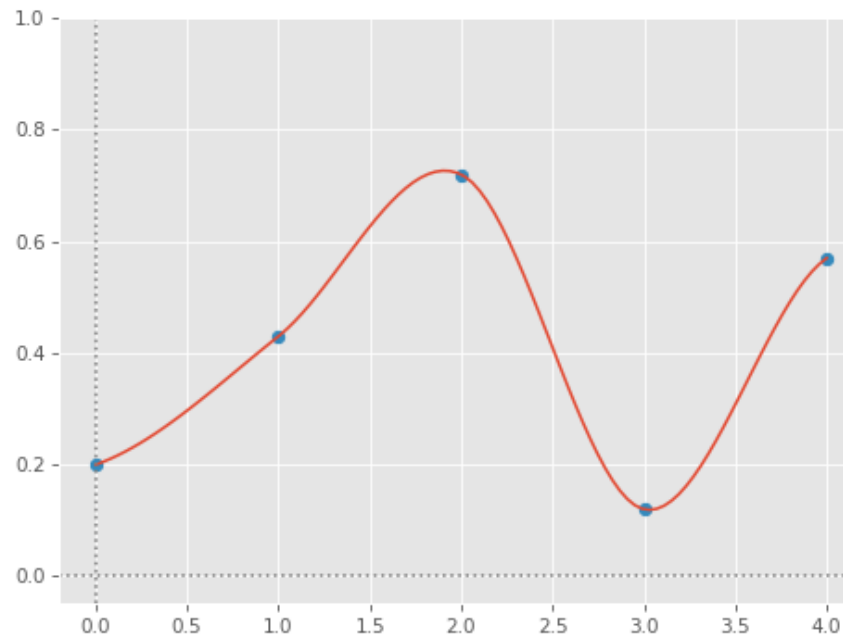
Hermite curve is good but what if we only have points, but no gradient?

Yes! Take four points instead of two, and use the central difference to "estimate" a gradient/velocity

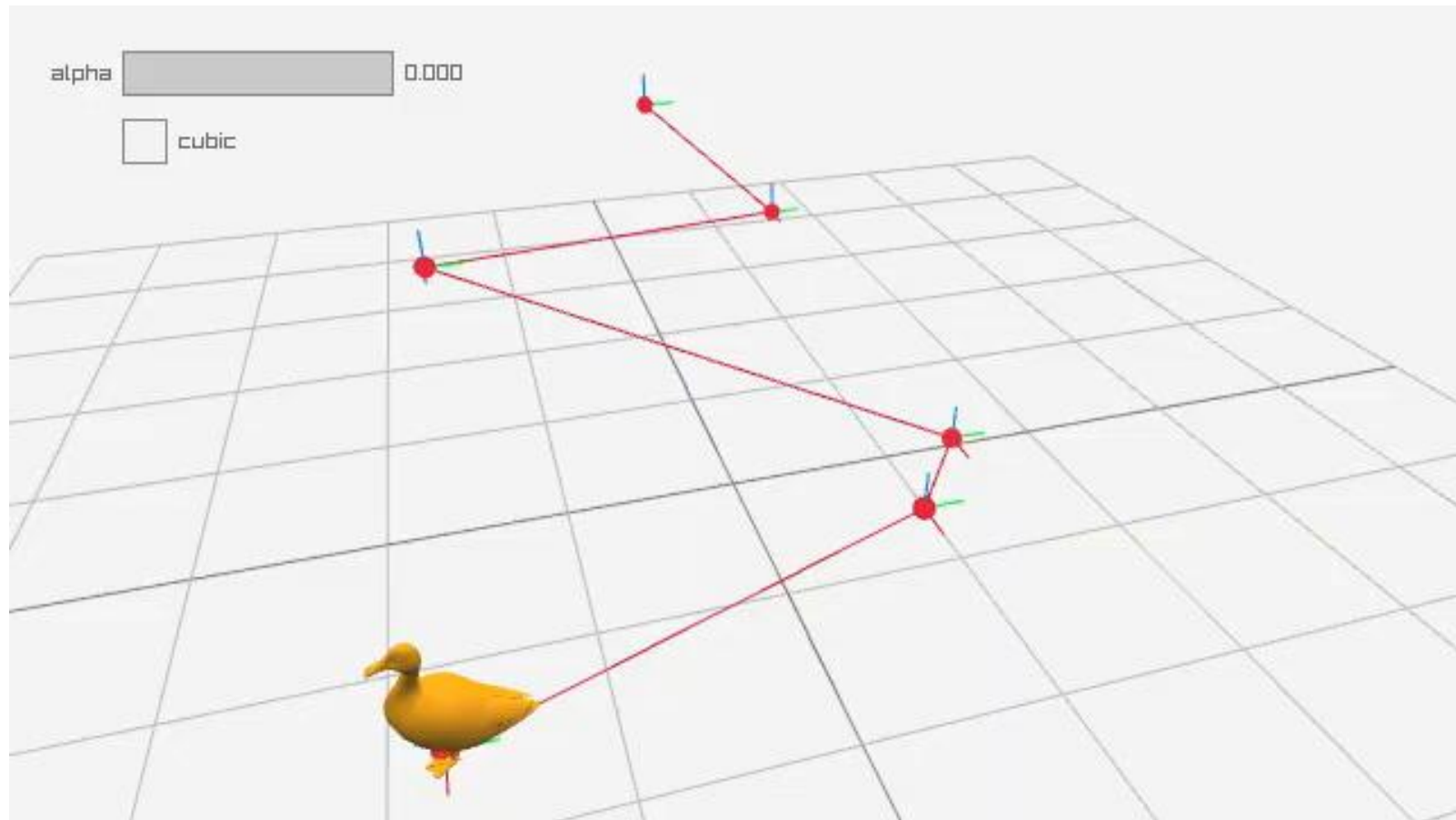


# Catmull-Rom Spline

```
void catmull_rom(  
    vec3& pos,  
    vec3& vel,  
    float x,  
    vec3 p0,  
    vec3 p1,  
    vec3 p2,  
    vec3 p3)  
{  
    vec3 v1 = ((p1 - p0) + (p2 - p1)) / 2;  
    vec3 v2 = ((p2 - p1) + (p3 - p2)) / 2;  
    hermite(pos, vel, x, p1, p2, v1, v2);  
}
```



# Comparing the linear and cubic interpolation



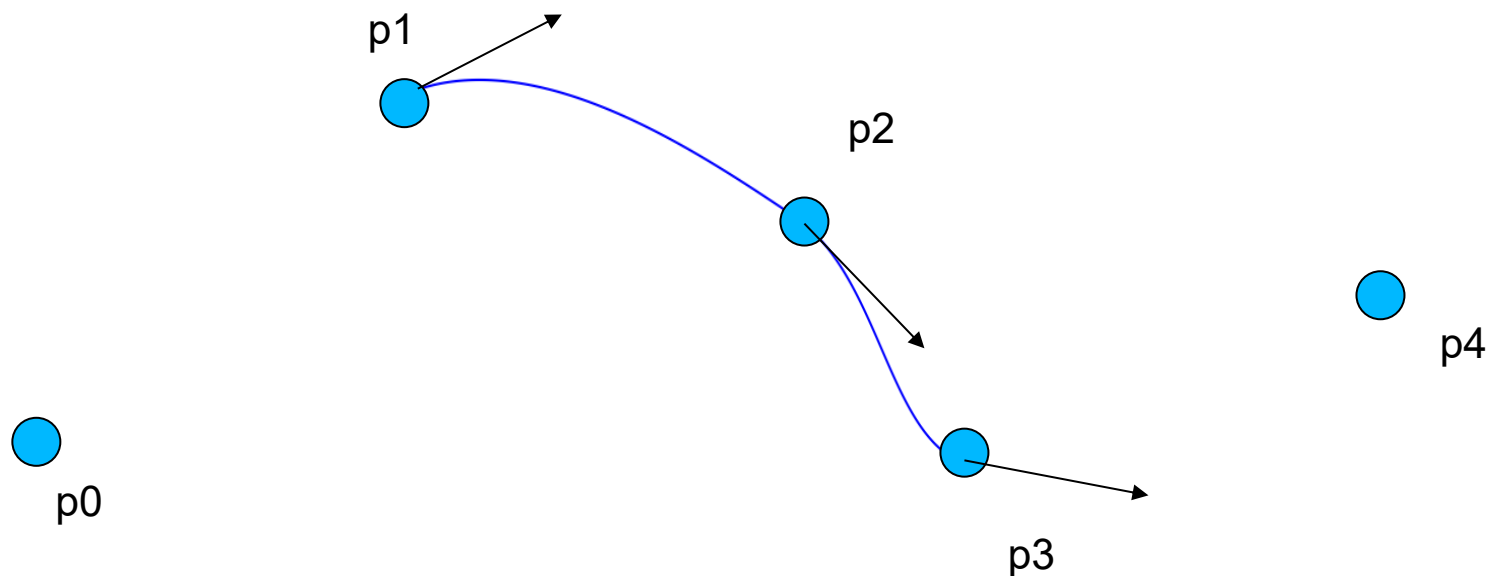
# Catmull-Rom Spline

$$p(u) = \begin{bmatrix} 1 & u & u^2 & u^3 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\tau & 0 & \tau & 0 \\ 2\tau & \tau - 3 & 3 - 2\tau & -\tau \\ -\tau & 2 - \tau & \tau - 2 & \tau \end{bmatrix} \begin{bmatrix} \mathbf{p}_{i-2} \\ \mathbf{p}_{i-1} \\ \mathbf{p}_i \\ \mathbf{p}_{i+1} \end{bmatrix}$$

Each segment defined by four points  $\mathbf{p}_{i-2}, \mathbf{p}_{i-1}, \mathbf{p}_i, \mathbf{p}_{i+1}$

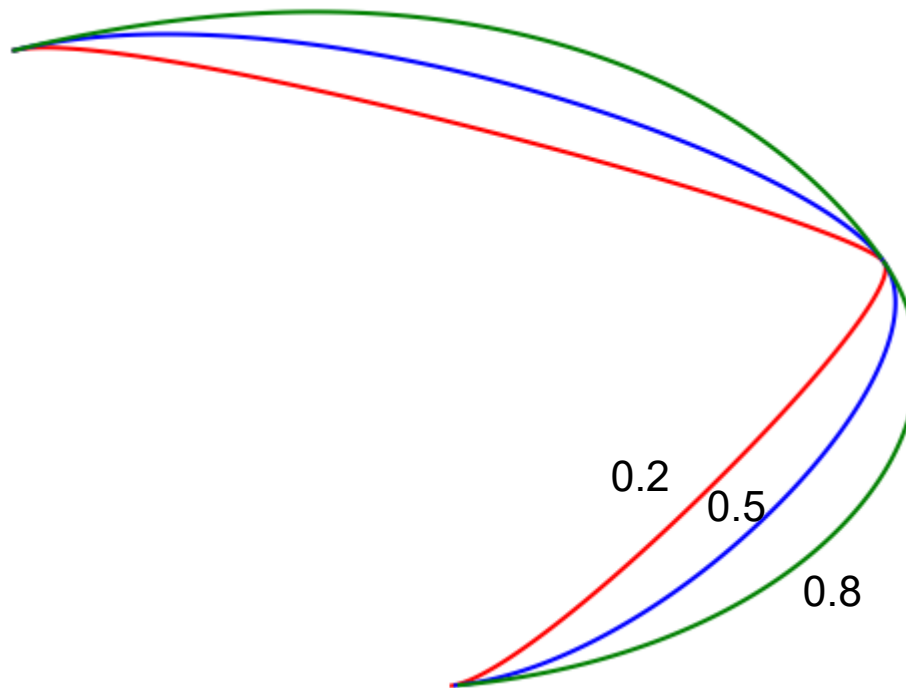
$u$  changes from 0 to 1 in each segment

$\tau$  is known as “tension” and it affects how sharply the curve bends at the (interpolated) control points



# Catmull-Rom Spline

Curves with different  $\tau$



# Quaternion Hermite Interpolation

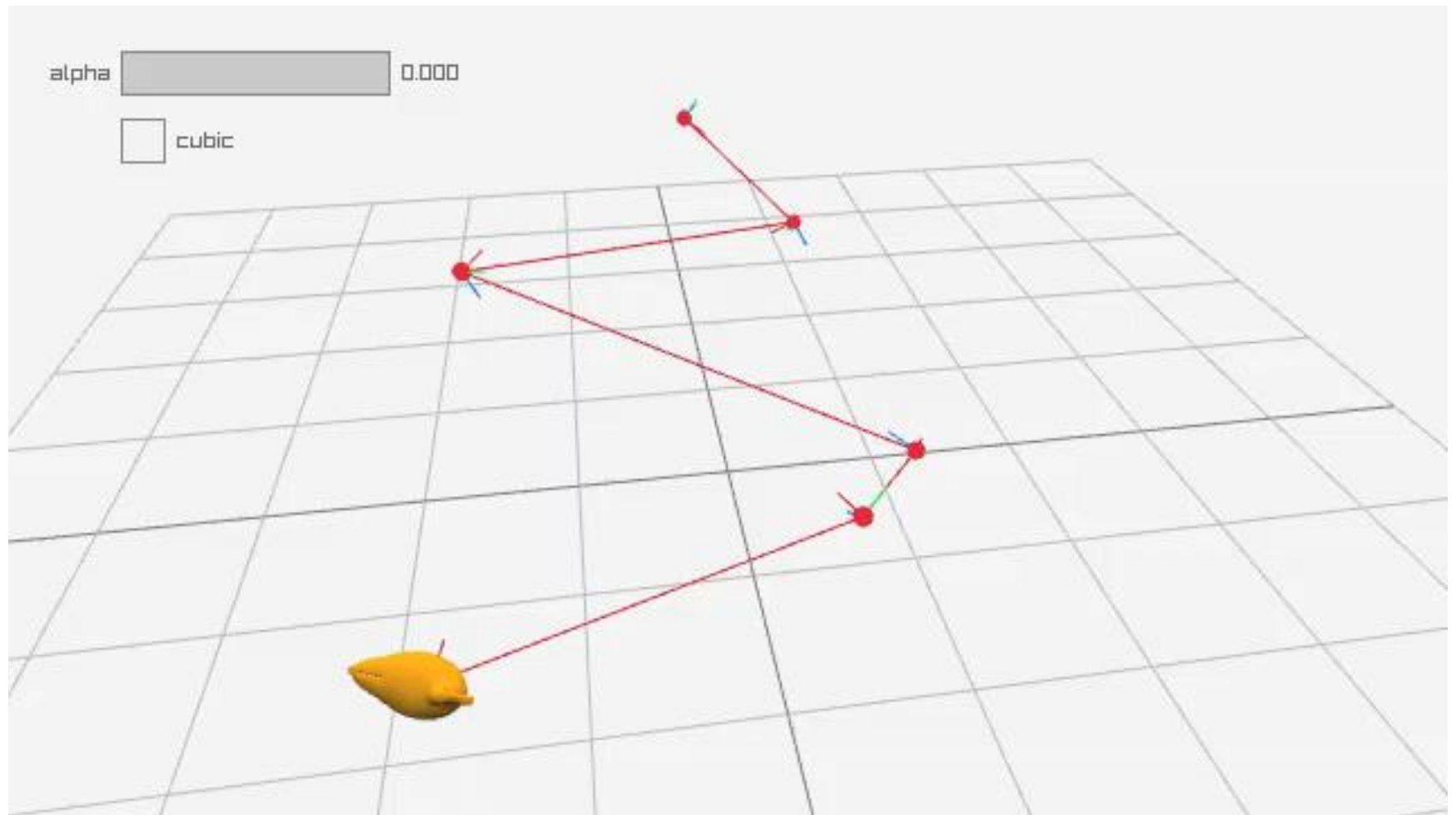
- Let's start from the Hermite interpolation again.
- This time, we give two quaternions and the angular velocities at the two ends.
- The angular velocity is in the angle-axis representation so need to do conversion when applying them to quaternions

```
void quat_hermite(  
    quat& rot,  
    vec3& vel,  
    float x,  
    quat r0,  
    quat r1,  
    vec3 v0,  
    vec3 v1)  
{  
    float w1 = 3*x*x - 2*x*x*x;  
    float w2 = x*x*x - 2*x*x + x;  
    float w3 = x*x*x - x*x;  
  
    float q1 = 6*x - 6*x*x;  
    float q2 = 3*x*x - 4*x + 1;  
    float q3 = 3*x*x - 2*x;  
  
    vec3 r1_sub_r0 = quat_to_scaled_angle_axis(quat_abs(quat_mul_inv(r1, r0)));  
  
    rot = quat_mul(quat_from_scaled_angle_axis(w1*r1_sub_r0 + w2*v0 + w3*v1), r0);  
    vel = q1*r1_sub_r0 + q2*v0 + q3*v1;  
}
```

# Quaternion Catmull-Rom Interpolation

- We can now make use of the Hermite interpolation and do Catmull-Rom interpolation on quaternions

```
void quat_catmull_rom(  
    quat& rot,  
    vec3& vel,  
    float x,  
    quat r0,  
    quat r1,  
    quat r2,  
    quat r3)  
{  
    vec3 r1_sub_r0 = quat_to_scaled_angle_axis(quat_abs(quat_mul_inv(r1, r0)));  
    vec3 r2_sub_r1 = quat_to_scaled_angle_axis(quat_abs(quat_mul_inv(r2, r1)));  
    vec3 r3_sub_r2 = quat_to_scaled_angle_axis(quat_abs(quat_mul_inv(r3, r2)));  
  
    vec3 v1 = (r1_sub_r0 + r2_sub_r1) / 2;  
    vec3 v2 = (r2_sub_r1 + r3_sub_r2) / 2;  
    quat_hermite(rot, vel, x, r1, r2, v1, v2);  
}
```

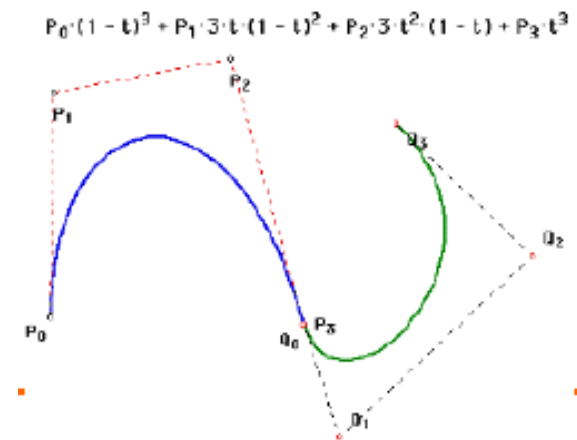
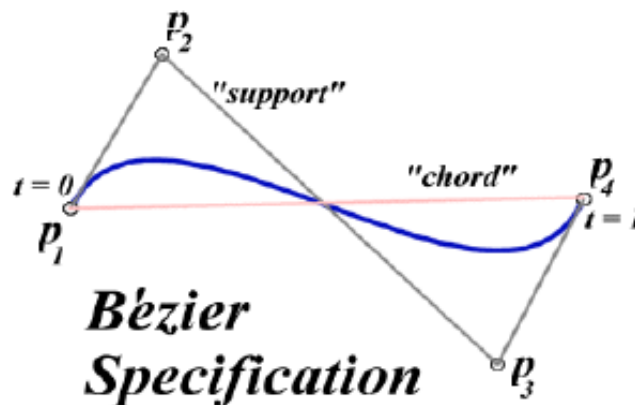


# Interpolation of the generalized coordinates

- Linear interpolation

$$\mathbf{p} = (1 - t)\mathbf{p}^i + t\mathbf{p}^{i+1}$$

- High-order polynomials (e.g. Bsplines, Bezier)  
Bezier: 2 end points, two points to control the tangent vector





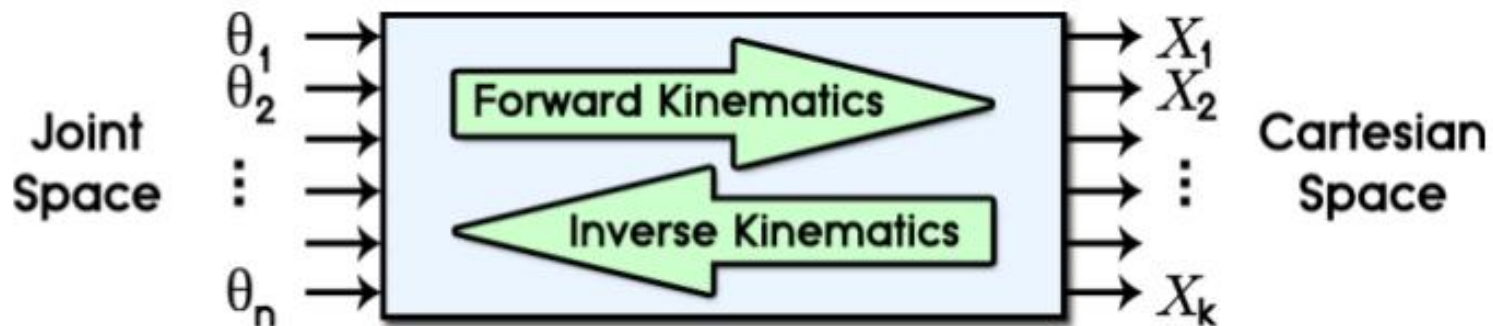
<https://www.youtube.com/watch?v=QAXOdj8mr7s>

# Problems with interpolating the generalized coordinates

- Important constraints might not be satisfied
  - The feet on the ground can slide
  - Mainly reasoned by interpolating the root wrong
    - “Error-propagation” along the hierarchy
- Solution
  - Specify the position of the joints and use IK to calculate the joint angles

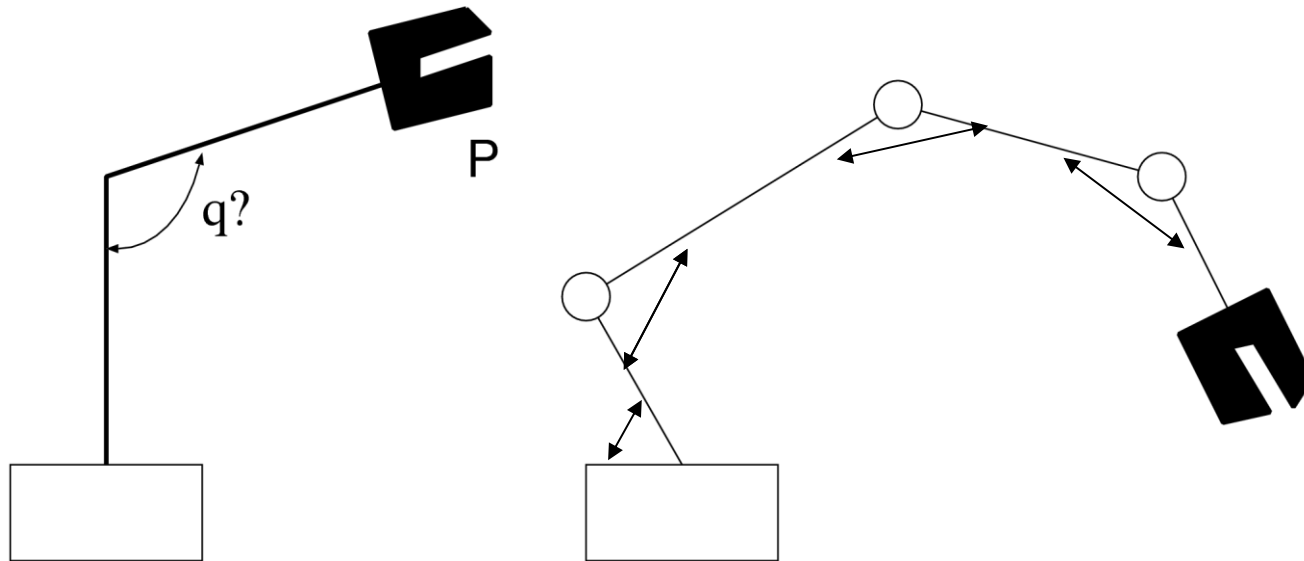
# Forward and Inverse Kinematics

- Forward kinematics solution is straightforward to compute and unique. However, hard to control.  
→ "We don't know the joint angles to reach a location."
- Inverse kinematics can have zero to infinite solutions. Solutions tell us the joint angles.



# Inverse Kinematics Problem

- Calculate the joint angles from the position of the end effector



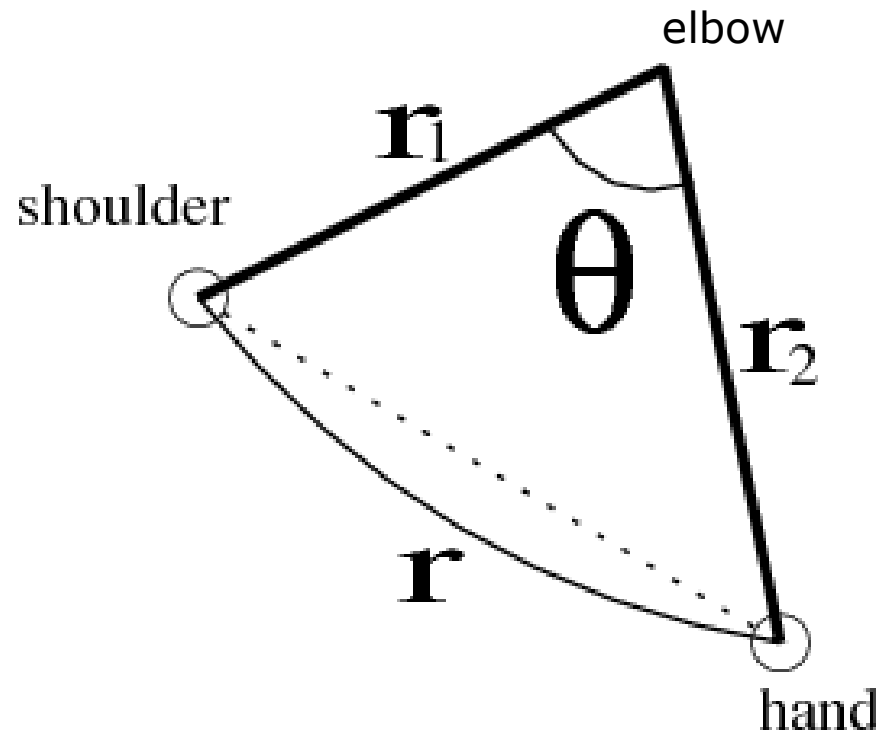
# Inverse Kinematics

- ❑ Forward Kinematics: calculating the joint positions from the joint angles
- ❑ **Inverse Kinematics : Calculate the joint angles based on the joint positions**
- Many different approaches
  - ❑ Analytical approaches (analytical solution exists)
  - ❑ CCD (Cyclic Coordinate Descent)
  - ❑ Jacobian-based methods (compute by optimization)
- Originated from robotics, but also essential in animation. Main difference: speed vs. accuracy

# Analytical Approaches

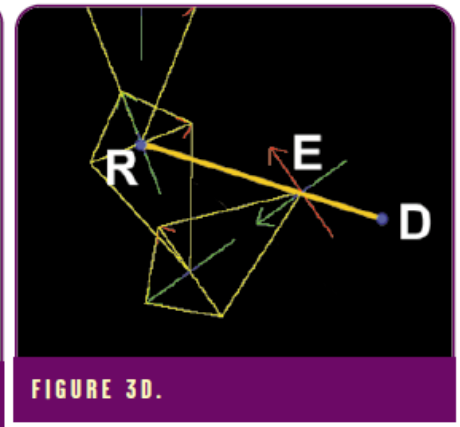
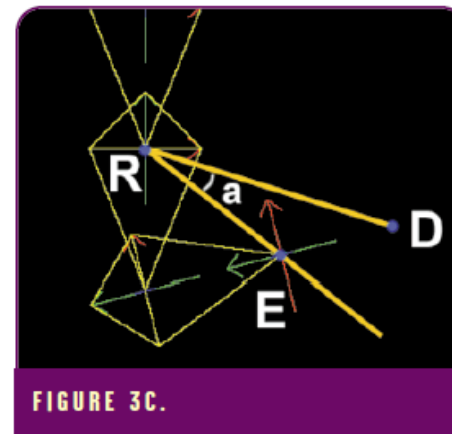
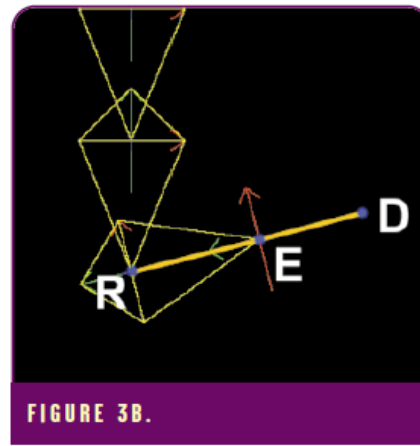
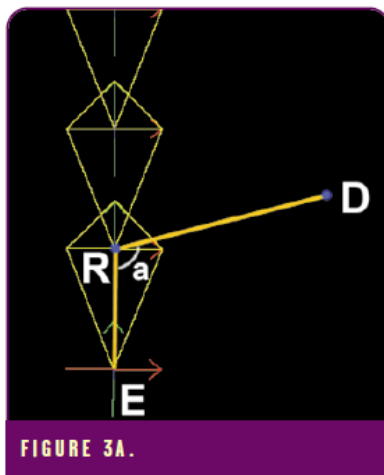
- Using an analytical solver for calculating the joint angles
- e.g. suppose the positions of the wrist and shoulder are given, calculate the elbow angle

$$\theta = \arccos \frac{r_1^2 + r_2^2 - r^2}{2 r_1 r_2}$$



# Cyclic-Coordinate Descent

- Moving the joints closest to the end effectors first and minimize the distance between the end effector and the target
- Move up the hierarchy and move the next joint to minimize the distance between the end effector and the target
- Repeat the process until the base is reached
- Move to the first joint again and repeat the same process until the end effector reaches the target or max iterations reached
  - This is needed to handle cases that the target is not reachable
  - 2D Tutorial:  
<http://www.ryanjuckett.com/programming/cyclic-coordinate-descent-in-2d/>



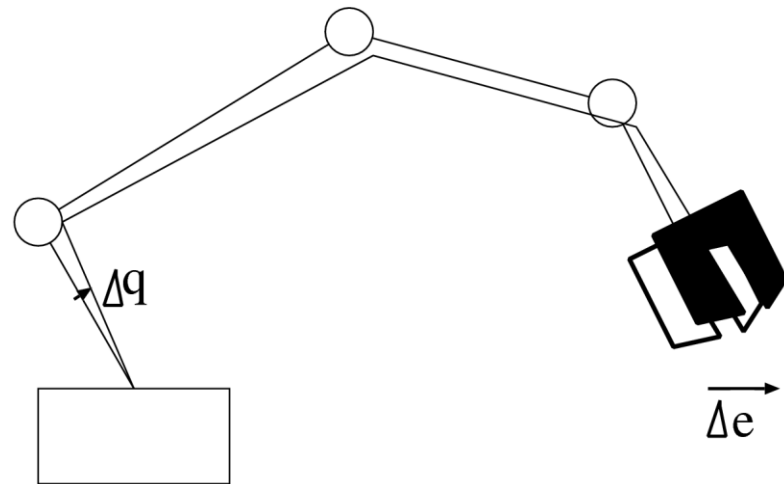
# Pseudo-inverse method

- The relationship between the end effector position and the joint angles are non-linear

$$\mathbf{e} = \mathbf{f}(\mathbf{q})$$

- But for small movements, their relation can be considered linear

$$\Delta \mathbf{e} = \mathbf{J} \Delta \mathbf{q}$$



# Jacobian matrix

- Correlates the movement of the end effector  $\Delta e$  with movements of the joints  $\Delta q_i$
- Each column describing how much  $\Delta e$  changes when the  $\Delta q_i$  is changed

$$\Delta e = J \Delta q$$

$$\text{Jacobian} \quad J = \frac{\partial e}{\partial q} \approx \begin{pmatrix} \frac{\Delta e_x}{\Delta q_1} & \dots & \frac{\Delta e_x}{\Delta q_n} \\ \frac{\Delta e_y}{\Delta q_1} & \dots & \frac{\Delta e_y}{\Delta q_n} \\ \frac{\Delta e_z}{\Delta q_1} & \dots & \frac{\Delta e_z}{\Delta q_n} \end{pmatrix}$$

# Pseudo-inverse method

- The relationship between the end effector position and the joint angles are non-linear

$$\mathbf{e} = \mathbf{f}(\mathbf{q})$$

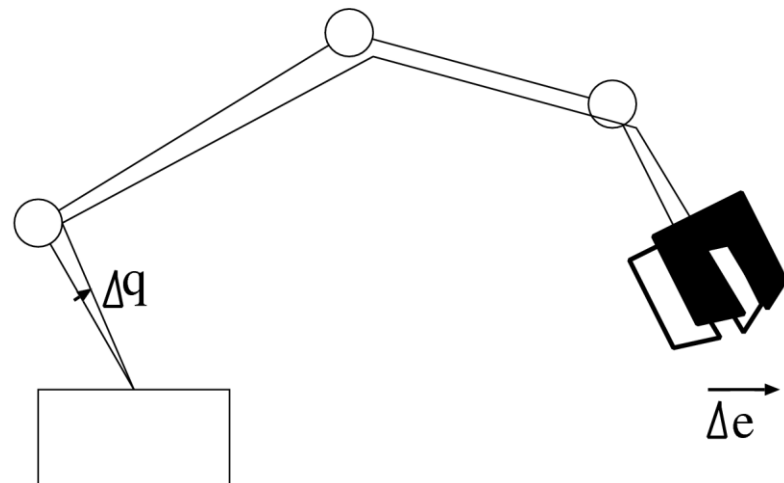
- But for small movements, their relation can be considered linear

$$\Delta \mathbf{e} = \mathbf{J} \Delta \mathbf{q}$$

This is Forward mapping

Inverse mapping?

$$\Delta \mathbf{e} \longrightarrow \Delta \mathbf{q}$$



# Pseudo Inverse Matrix

- The minimal joint angle movements that achieves the end effector movement  $\Delta \mathbf{e}$  can be computed by the pseudo inverse matrix

pseudo inverse  $\mathbf{J}^+ = \mathbf{J}^T (\mathbf{J}\mathbf{J}^T)^{-1}$

$$\Delta \mathbf{q} = \mathbf{J}^+ \Delta \mathbf{e}$$

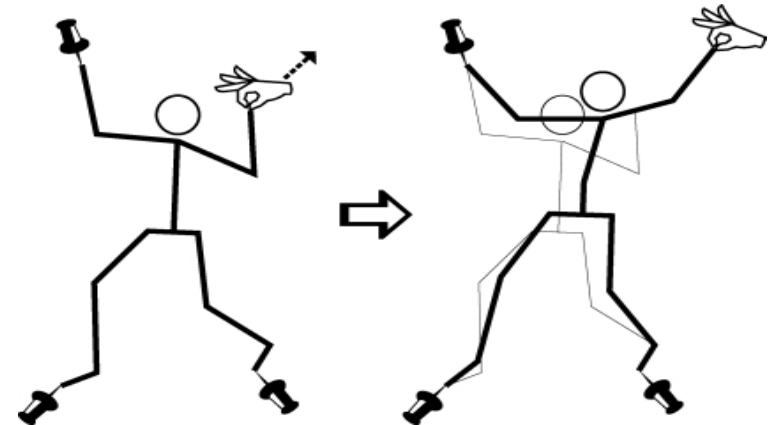
# Pseudo-inverse Method

Iteratively updating the generalized coordinates so that the position constraints are satisfied

$\mathbf{e}$ : end effector position

$\mathbf{g}$ : target location

```
while (  $\Delta \mathbf{e}$  is too far from  $\mathbf{g}$  ) {  
    compute the Jacobian matrix  $\mathbf{J}$   
    compute the pseudoinverse of the Jacobian matrix  $\mathbf{J}^+$   
    compute change in joint DOFs:  $\Delta \mathbf{q} = \mathbf{J}^+ \Delta \mathbf{e}$   
    apply the change to DOFs, move a small step of  
         $\mathbf{q} = \mathbf{q} + \Delta \mathbf{q}$   
}
```

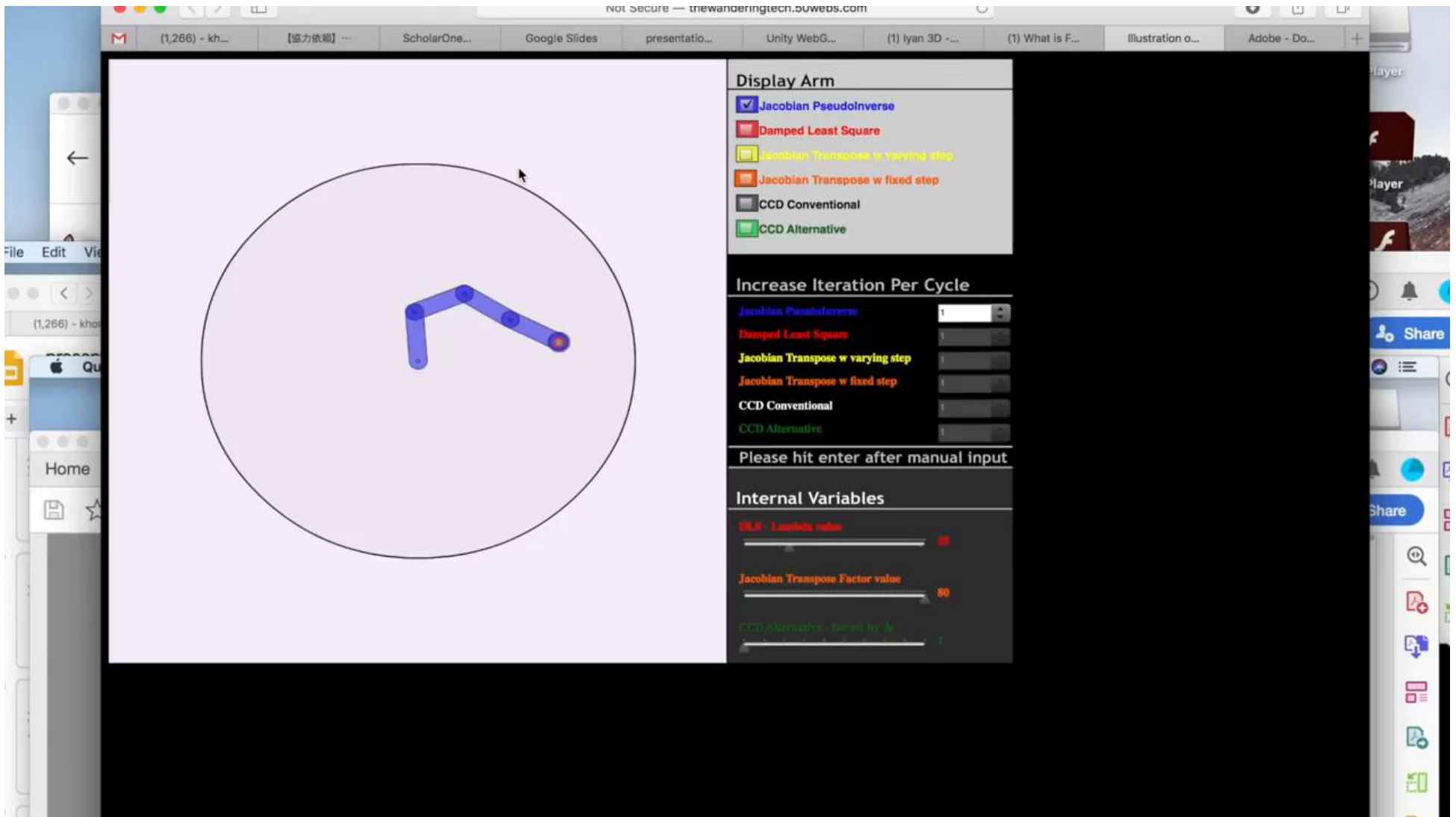


$$\Delta \mathbf{e} = \mathbf{J} \Delta \mathbf{q}$$

# Singularity Problem

- There can be postures that the end effector cannot be moved to some directions
  - i.e. when all the joints are fully extended
- The system becomes unstable
- Solution: **Damped Least Squares**  
(imposing soft constraints):

$$\mathbf{J}^* = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T + k^2 \mathbf{I})^{-1}$$



# Damped Least Squares

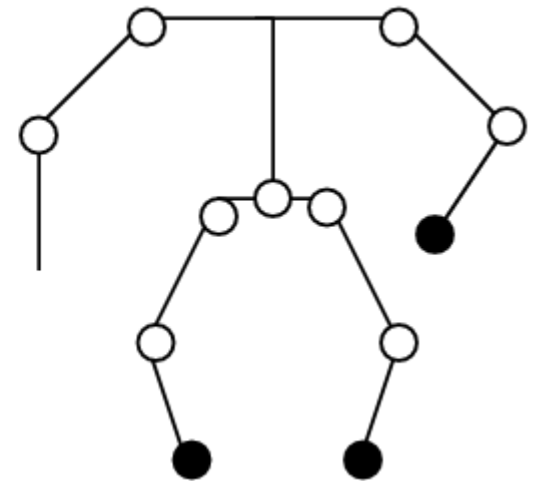
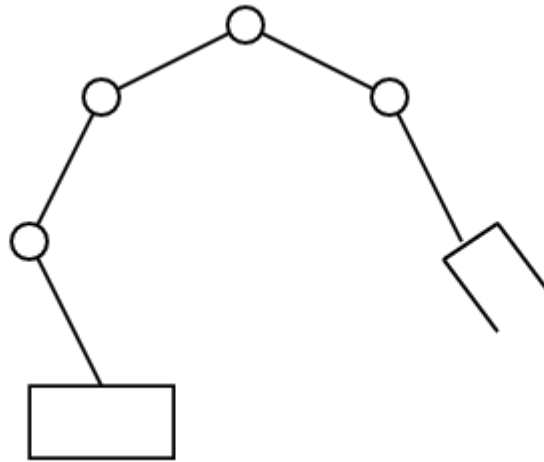
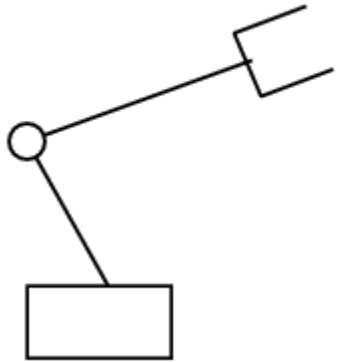
Instead of minimizing

$$g(\dot{\mathbf{q}}, \boldsymbol{\lambda}) = \frac{1}{2} \dot{\mathbf{q}}^T \dot{\mathbf{q}} + \boldsymbol{\lambda}^T (\mathbf{v}_e - \mathbf{J} \dot{\mathbf{q}}),$$

minimize

$$g''(\dot{\mathbf{q}}) = \frac{1}{2} k^2 \dot{\mathbf{q}}^T \dot{\mathbf{q}} + \frac{1}{2} (\mathbf{v}_e - \mathbf{J} \dot{\mathbf{q}})^T (\mathbf{v}_e - \mathbf{J} \dot{\mathbf{q}})$$

# Which method will be good for controlling each of the below?



# Cons and Pros of IK

- ❑ Analytical : Fast, but for most cases have no analytical solution
- ❑ CCD : simple, fast, easy to implement, can take into account the limit of the joint angles, may have oscillation problems
- ❑ Pseudo Inverse:
  - Used in robotics often, can handle any topological structure, multiple constraints
  - Can incorporate physics
  - Singularity problems (unstable when the limb is fully extended) -> damped least squares

# Summary

- Representation of the Posture
  - Euler angles, generalized coordinates, quaternions
- Character Animation by Interpolation
- Inverse Kinematics
  - Analytical
  - CCD
  - Jacobian Pseudoinverse
  - Damped-Least-Squares

# Links, Readings

- ❑ Poser <http://poser.smithmicro.com/poser.html>
- ❑ MikuMikuDance
  - ❑ <https://sites.google.com/view/vpvp/>
- ❑ About CCD [http://graphics.cs.cmu.edu/nsp/course/15-464/Fall09/assignments/asst2/jlander\\_gamedev\\_nov98.pdf](http://graphics.cs.cmu.edu/nsp/course/15-464/Fall09/assignments/asst2/jlander_gamedev_nov98.pdf)
- ❑ IK introduction
  - <http://www.math.ucsd.edu/~sbuss/ResearchWeb/ikmethods/iksurvey.pdf>
- ❑ Quaternion guide: <https://graphics.stanford.edu/courses/cs348a-17-winter/Papers/quaternion.pdf>
- ❑ A robotics textbook (see chapter 3)
  - <http://link.springer.com/content/pdf/10.1007%2F978-1-84628-642-1.pdf>